

Inhaltsverzeichnis

A Einleitung	1
A.1 Praktikums-Modus	1
A.2 Aufgabendurchführung	2
A.3 Baukasten	2
A.4 Protokoll	3
B Grundlagen	6
B.1 Multimeter	6
B.2 Steckbrett	7
B.3 Farbcodierung von Widerständen	8
B.4 Widerstandsreihen	9
C Arduino® Programmierung - die Basics	10
C.1 Arduino® IDE	10
C.2 Programmgerüst	11
C.3 Grundlegende Befehle und Funktionen	12
C.4 Serial Monitor	13
C.5 Interrupts	14
C.6 Verwendung von Tastern	15
C.7 Hilfe während des Praktikums	17
1 Ampelsteuerung	18
1.1 Vorbereitung	18
1.2 Praktikumsaufgabe	19
1.3 Checkliste für das Protokoll	19
2 Reaktionsspiel	20
2.1 Vorbereitung	20
2.2 Praktikumsaufgabe	22
2.3 Checkliste für das Protokoll	22
3 Realisierung eines Logic Analyzers	23
3.1 Vorbereitung	23
3.2 Praktikumsaufgabe	24
3.3 Checkliste für das Protokoll	25
4 Wertebestimmung von Widerstand und Kondensator	26
4.1 Vorbereitung	26
4.2 Praktikumsaufgabe	29
4.3 Checkliste für das Protokoll	30
5 Ansteuerung eines Lautsprechers	31
5.1 Vorbereitung	35
5.2 Praktikumsaufgabe	37
5.3 Checkliste für das Protokoll	38
6 Motorsteuerung mit Näherungssensor	39
6.1 Vorbereitung	44
6.2 Praktikumsaufgabe	46
6.3 Checkliste für das Protokoll	47

7	Datenübertragung mittels Infra-Rot (IR)	48
7.1	Vorbereitung	49
7.2	Praktikumsaufgabe	52
7.3	Checkliste für das Protokoll	53

A Einleitung

In diesem Skript zum *Praktikum Digitale Schaltungen* für InformatikerInnen werden ausgehend von den theoretischen Grundkenntnissen aus den Vorlesungen *Elektronik* und *Digitale Schaltungen*, die darüber hinausgehenden notwendigen Kenntnisse für die Durchführung des Praktikums vermittelt. In Kapitel B werden Grundlagen zu den einzelnen Messgeräten und Komponenten, die während des Praktikums verwendet werden, gezeigt. Kapitel C vermittelt im Speziellen einfache Grundlagen der Arduino-Programmierung, welche im Praktikum umgesetzt werden sollen. Die nachfolgenden Kapitel 1 bis 7 beinhalten Praktikumsaufgaben, die innerhalb des Praktikums bearbeitet werden sollen. Jede Teilaufgabe setzt sich aus den Abschnitten *Allgemeines*, *Vorbereitung* und *Durchführung* zusammen.

A.1 Praktikums-Modus

A.1.1 Notengebung

Die LVA *Praktikum Digitale Schaltungen* ist in zwei Teilbereiche unterteilt, welche zur Notenbildung beitragen:

- Praktikumsdurchführung in Kleingruppen
- Protokoll

Während der Praktikumsdurchführung wird Ihre Mitarbeit durch kurze mündliche Zwischenfragen zur aktuellen Aufgabenstellung oder zur Vorbereitung bewertet. Diese Bewertung ist Teil der Gesamtnote.

Der Abschluss der Praktikums bildet die Abgabe eines Praktikumsprotokolls pro Kleingruppe. Das Protokoll soll nachvollziehbar alle Arbeitsschritte zur Lösungen der gestellten Aufgaben beinhalten. Die Punkte der bewerteten Protokolle werden für die gesamte Gruppe vergeben und werden ebenfalls anteilig zur Gesamtnote gerechnet.

A.2 Aufgabendurchführung

Das Praktikum selbst findet an vier Tagen (ganztägig) statt. Während des Praktikums arbeiten Sie in Kleingruppen zusammen. Die Gruppeneinteilung kann eigenständig gewählt werden. Die genauen Termine, sowie die Gruppeneinteilung dazu, entnehmen Sie bitte dem KUSSS. Während dieser Tage sollen alle, im Skript enthaltenen, Aufgaben durchgearbeitet werden. Achten Sie dabei darauf alle Fragen aus den Aufgabenstellungen zu bearbeiten und Ihre Arbeitsschritte zu dokumentieren. Die einzelnen Aufgabenblöcke bestehen jeweils aus 2 Teilabschnitten:

- **Vorbereitung:**

Alle unter diesem Abschnitt gestellte Fragen und Aufgabenstellungen dienen als Vorbereitung auf das Praktikum bzw die Aufgabe und können daher vor den Praktikumstagen gelöst werden. Die Ergebnisse dieser Vorbereitung sind schriftlich auszuarbeiten.

- **Praktikumsaufgabe:**

Die Aufgaben aus diesem Abschnitt sollen während der Praktikumstage Schritt für Schritt abgearbeitet werden. Die Aufgabenstellung wurde dabei so gewählt, dass die aufeinander folgenden Beispiele so weit wie möglich aufeinander aufbauen. Es ist daher sinnvoll, die Aufgaben nach der, im Skript angegebenen Reihenfolge zu lösen. Wurde eine Teilaufgabe während des Praktikums gelöst, melden Sie dies bitte dem jeweiligen Tutor, der jeweiligen Tutorin oder einem/einer PraktikumsbetreuerIn. Die Korrektheit der Lösung wird dann von ihm/ihr überprüft. Für nicht gelöste Aufgaben werden für die gesamte Kleingruppe Punkte von der Gesamtnote abgezogen.

- **Checkliste für das Protokoll:**

Diese Liste soll als Richtlinie gelten, welche Informationen für das Protokoll relevant sind. Die angeführten Punkte stellen das absolute Minimum an Dokumentation dar, die in der Abgabe vorhanden sein sollte.

A.3 Baukasten

An den Praktikumstagen erhalten Sie einen Baukasten (siehe Abbildung 1), in dem alle elektronischen Bauteile, ein Mikrocontroller-Board, das in Abschnitt B beschriebene Steckbrett usw. enthalten sind. Pro Gruppe steht für die Dauer des Praktikums ein Baukasten zur Verfügung.



Abbildung 1: Praktikums-Baukasten mit allen relevanten Bauteilen für die einzelnen Aufgaben

Dem Baukasten liegt eine Liste bei, auf der alle enthaltenen Bauelemente aufgelistet sind. Überprüfen Sie bitte am Ende jedes Praktikumstages, ob alle Bauteile vorhanden sind. Sollten Bauteile während des

Praktikums verloren oder kaputt gehen, geben Sie bitte dem Praktikumsleiter/der Praktikumsleiterin Bescheid! Ersatzteile sind vorhanden.

A.4 Protokoll

Mit Hilfe des Protokolls soll es möglich sein, auch nach einem längeren Zeitraum die Messungen bzw. Aufgabenstellungen, unter den gleichen Randbedingungen, erneut durchführen zu können. Dabei ist es nötig, Umgebungsbedingungen, verwendete Messgeräte, Ablauf, Berechnungen usw. zu dokumentieren. Die folgenden Richtlinien stellen die Mindestanforderung an das Praktikumsprotokoll dar. Sollte ein Protokoll diesen Richtlinien nicht entsprechen werden dementsprechend Punkte abgezogen.

A.4.1 Abgabe

Das Protokoll ist spätestens 4 Wochen nach dem letzten Praktikumstermin als PDF abzugeben.

A.4.2 Deckblatt

Das Deckblatt des Protokolls muss folgende Informationen enthalten:

- LVA Bezeichnung
- Name und Matrikelnummer aller Beteiligten

A.4.3 Inhalt des Protokolls

Das Protokoll muss mindestens folgende Punkte beinhalten:

- Aufgabenstellung: Was soll umgesetzt werden? Was ist gegeben?
- Theoretische Betrachtung (Vorbereitende Dimensionierung, Berechnungen, Algorithmen ...), alle Ergebnisse und Antworten aus dem Abschnitt Vorbereitung!
- Messaufbau (die aufgebaute Schaltung unter Verwendung der genormten Schaltzeichen, die Schaltung muss mit dem tatsächlichen Messaufbau übereinstimmen, Pinnummern der Bauteile müssen erkennbar sein)
- Verwendete Prüflinge (DUT)
- Verwendete Messmethoden, Erklärung was?, wie? und warum so und nicht anders? gemessen wurde
- Verwendete Bauteile und deren Wert (z.B. Widerstand $R_1 = 100\ \Omega$, OPAMP1 = LM741)
- Programmlistings mit ausführlichen Kommentaren
- Messtabellen, Diagramme, Kurven, etc. (mit Einheiten!)
- Angabe der verwendeten und nicht allgemein bekannten Formelzeichen
- Auswertung von Berechnungen unter Angabe der verwendeten Formeln
- Fehlerdiskussion
- Zusammenfassung, Erklärungen (speziell wenn unerwartete Ergebnisse aufgetreten sind, soll versucht werden diese zu interpretieren)

A.4.4 Abbildungen

Bilder müssen groß genug eingefügt werden. Der Inhalt muss gut erkennbar bzw. leicht lesbar sein. Sollte Information durch Farben ausgedrückt werden, so ist auch bei der Abgabe auf einen dementsprechenden Druck zu achten. Bilder müssen eine Nummerierung und eine Bildunterschrift aufweisen.

A.4.5 Diagramme

Jedes Diagramm muss folgende Kriterien erfüllen:

- Es müssen mindesten zwei beschriftete Achsen vorhanden sein. Die Beschriftung besteht aus dem Namen der physikalischen Größe die aufgetragen wird, sowie deren Einheit. Die Einheit der Größe wird dabei in Klammern () geschrieben.
- Ein Pfeil auf der Achse gibt in welche Richtung die aufgetragene Größe zunimmt.
- Werden mehrere Kurven in ein Diagramm gezeichnet, so müssen diese eindeutig beschriftet sein. Weiters müssen die Kurven auch visuell unterscheidbar sein. Sollten Sie Farben für diese Unterscheidung verwenden, achten Sie darauf das Protokoll auch in Farbe für die Abgabe zu drucken!
- Wie auch Abbildungen (siehe Abschnitt A.4.4), sollten auch Diagramme mit einer Bildunterschrift und einer Nummerierung versehen werden.

Abbildung 2 zeigt ein korrektes Diagramm in dem alle Richtlinien beachtet wurden:

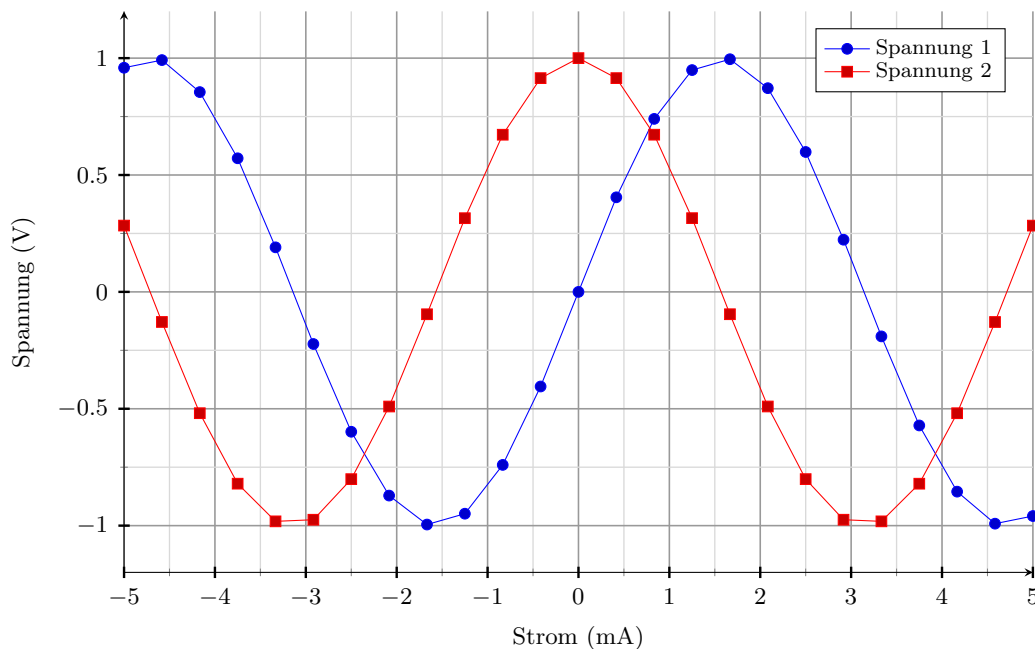


Abbildung 2: Beispiel wie ein korrektes Diagramm aussehen sollte

A.4.6 Tabellen

Es muss klar erkenntlich sein welche Daten in der Tabelle dargestellt werden, dies kann durch eine Beschriftung über der Tabelle erfolgen. Physikalische Größen müssen wie in Abschnitt A.4.5 beschrieben immer mit Einheit angegeben werden. Die Einheit wird dabei wieder in () geschrieben. Ein Beispiel für eine korrekte Darstellung von Daten in einer Tabelle finden Sie in Tabelle 1.

Tabelle 1: Strom und Spannung zu verschiedenen Zeitpunkten

Zeit t (s)	Spannung U_1 (V)	Strom I_1 (mA)
0	1.23	12.2
1	1.45	14.0
2	1.12	13.8
3	0.8	12.9
4	0.23	12.7
5	0.0	12.3

A.4.7 Schaltpläne

Beim Zeichnen von Schaltplänen sind grundsätzlich Schaltzeichen der Norm Din EN 60617-4 zu verwenden. Eine gute Übersicht der in Europa üblichen Symbole ist unter www.wikipedia.org/wiki/Schaltzeichen zu finden. Prinzipiell gilt aber:

- Eine Kreuzung von Leitungen, die elektrisch verbunden sind, ist mit einem Punkt zu kennzeichnen.
- Alle Symbole müssen einen eindeutigen Namen haben.
- Es muss pro Schaltplan mindesten ein GND-Symbol vorhanden sein.

In Abbildung 3 ist ein Beispiel eines korrekten Schaltplans ersichtlich.

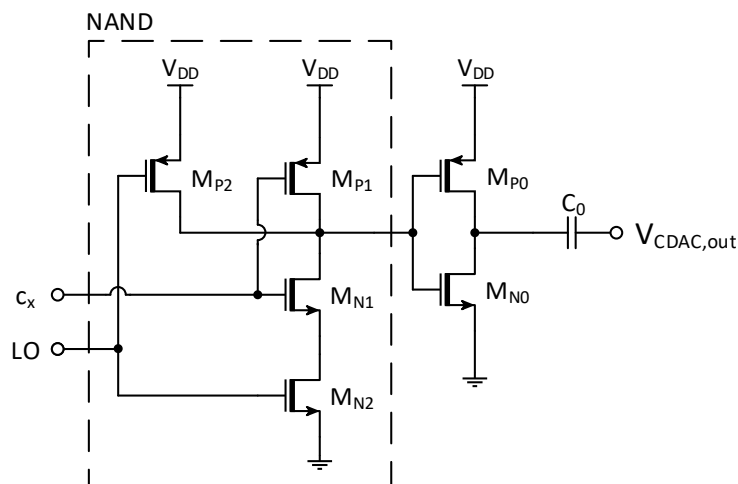


Abbildung 3: Beispiel eines korrekten Schaltplans

B Grundlagen

Im folgenden Kapitel sollen einige wesentliche Grundlagen für die Praktikumsdurchführung, sowie die verwendeten Messgeräte erklärt werden.

B.1 Multimeter

Im Praktikum steht Ihnen für das Messen von Strom, Spannung und Widerstandswerten ein Tischmultimeter der Firma Keithley zur Verfügung. In Abbildung 4 ist die Frontansicht des Messgerätes ersichtlich.



Abbildung 4: Tischmultimeter - Frontansicht

Folgende Einstellungen müssen zum Messen getroffen werden:

- Spannungsmessung:
Zur Spannungsmessung muss mittels Knopfdruck auf **DCV** die Messgröße auf Gleichspannung gestellt werden. Als Eingänge werden für diese Messung die beiden rechten (rot und schwarz) Eingänge des Multimeters verwendet, welche mit **Input** (HI und LO) gekennzeichnet sind.
- Strommessung:
Zur Strommessung muss mittels Knopfdruck auf **DCI** die Messgröße auf Gleichstrom umgestellt werden. Als Eingänge werden für diese Messung der rechte mit **LO** gekennzeichnete (schwarz) Eingang, sowie der rechts unten befindliche (weiße) **AMPS** Eingang verwendet.
- Widerstandsmessung:
Zur Messung von Widerständen muss mittels Knopfdruck auf **$\Omega 2$** die Messgröße auf Widerstand gestellt werden. Als Eingänge werden für diese Messung die beiden rechten (rot und schwarz) Eingänge des Multimeters verwendet, welche mit **Input** (HI und LO) gekennzeichnet sind.

B.2 Steckbrett

Ein Steckbrett dient dazu, ohne Bauteile verlöten zu müssen und ohne großen zeitlichen Aufwand, elektronische Schaltungen aufzubauen und messen zu können. Es wird oft verwendet um kleinere Schaltungsteile zu testen oder Prototypen von Schaltungen aufzubauen. Im Praktikum wird das Steckbrett verwendet um Schaltungen, welche zusätzlich zum Arduino® Board benötigt werden, aufzubauen, zu testen und zu messen. Das im Praktikum zur Verfügung gestellte Steckbrett ist in Abbildung 5 ersichtlich.



Abbildung 5: Steckbrett mit markierten leitenden Verbindungen

Einige der Stecklöcher sind intern miteinander leitend verbunden. In Abbildung 5 sind beispielhaft die Verbindungen für die ersten Reihen und Spalten mit grünen Linien gekennzeichnet. Die Leisten am oberen und unteren Rand des Steckbrettes können für die Versorgungsspannung V_{DD} , sowie für den Masseanschluss GND verwendet werden. Diese Leisten sind (wie ebenfalls in Abbildung 5 markiert) über die halbe Länge des Steckbrettes hindurch elektrisch leitend verbunden.

B.2.1 Schaltplan am Steckbrett realisieren

Das folgende kleine Beispiel soll die Realisierung eines gegebenen Schaltplanes auf einem Steckbrett zeigen. In Abbildung 6a ist ein Schaltplan ersichtlich, der auf einem Steckbrett aufgebaut werden soll. Die Schaltung besteht aus einem Arduino® Mikrocontroller sowie 3 Widerständen, die einen Spannungsteiler zwischen $V_{DD} = 5\text{ V}$ und GND bilden. Der Arduino® wird in diesem Beispiel rein als Spannungsversorgung verwendet.

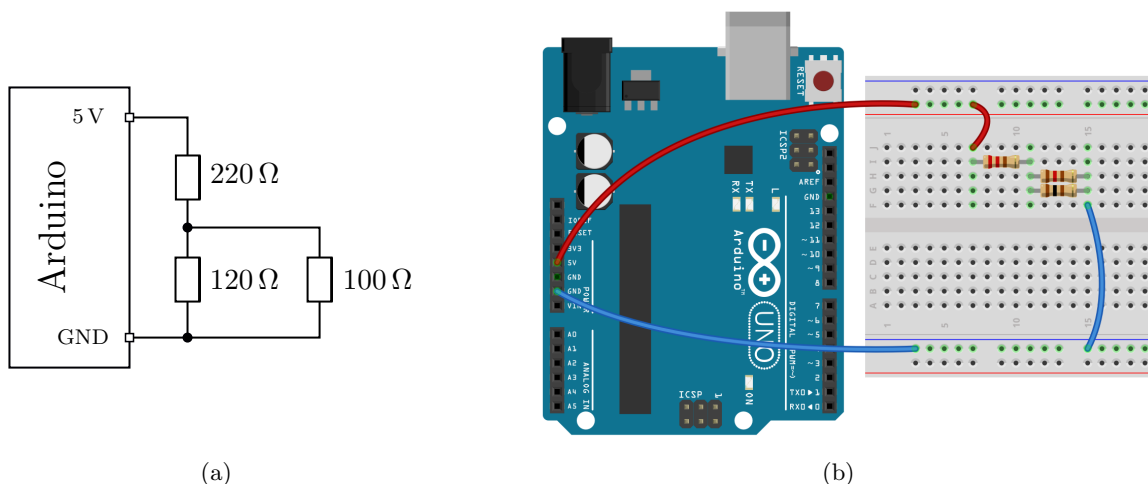


Abbildung 6: (a) Schaltplan und (b) Steckbrett-Realisierung einer einfachen Schaltung

In Abbildung 6b ist dieselbe Schaltung am Steckbrett aufgebaut. Die miteinander verbundenen Reihen und Spalten sind in der Abbildung grün markiert.

B.3 Farbcodierung von Widerständen

Die Widerstände, welche im Praktikum verwendet werden, sind mit einer Farbcodierung versehen. Anders als bei Widerständen in SMD-Gehäusen (surface-mounted-device) kann bei dieser Art von Widerständen aufgrund der runden Bauteilform, der elektrische Wert (in Ω) nicht direkt auf das Bauteil gedruckt werden. Daher werden Farbringe rund um die zylindrischen Widerstände gedruckt, die den Widerstandswert codieren.

Es gibt Farbcodes mit 3, 4, 5 oder 6 Ringen. Für 3 bzw. 4 Ringe gilt folgendes Schema: die ersten beiden Ringe geben einen zweistelligen Wert von 10Ω bis 99Ω an. Der dritte Ring gibt einen Multiplikator in Zehnerpotenzen von 10^{-2} bis 10^9 an. Ist ein vierter Ring vorhanden, so wird damit die Toleranzklasse angegeben. Eine Klasse von $\pm 5\%$ bedeutet, dass der angegebene Widerstandswert um $\pm 5\%$ von Bauteil zu Bauteil schwanken kann. Bei 5 oder 6 Farbringen wird zusätzlich zu 10er und 1er Stelle eine 100er Stelle angegeben (ersten 3 Ringe), danach folgt wieder der Multiplikator. Der sechste Ring gibt den Temperaturkoeffizienten des Widerstandes an.

Die Ableserichtung wird auf unterschiedliche Arten festgelegt. Meist wird der letzte Farbring mit einem größeren Abstand zu den restlichen Ringen gedruckt. Eine andere Variante ist, dass der erste Ring einen kleineren Abstand vom Rand des Widerstandskörpers hat als der letzte Ring (alle Ringe sind also nach links gerückt). Ob die Ringe richtig abgelesen wurden lässt sich meist leicht überprüfen, da bei fast allen Widerständen die jeweils andere Leserichtung keine sinnvollen Werte ergibt. Die Farbcodierung für 4 Ringe ist in Tabelle 2 ersichtlich:

Tabelle 2: Farbkodierung von Widerständen mit 4 Ringen

Farbe		Widerstandswert in Ω			Toleranz
		1. Ring (10er Stelle)	2. Ring (1er Stelle)	3. Ring (Multiplikator)	4. Ring
keine	X	-	-	-	$\pm 20\%$
silber		-	-	$10^{-2} = 0.01$	$\pm 10\%$
gold		-	-	$10^{-1} = 0.1$	$\pm 5\%$
schwarz		-	0	$1 = 1$	-
braun		1	1	$10^1 = 10$	$\pm 1\%$
rot		2	2	$10^2 = 100$	$\pm 2\%$
orange		3	3	$10^3 = 1\,000$	-
gelb		4	4	$10^4 = 10\,000$	-
grün		5	5	$10^5 = 100\,000$	$\pm 0.5\%$
blau		6	6	$10^6 = 1\,000\,000$	$\pm 0.25\%$
violett		7	7	$10^7 = 10\,000\,000$	$\pm 0.1\%$
grau		8	8	$10^8 = 100\,000\,000$	$\pm 0.05\%$
weiß		9	9	$10^9 = 1\,000\,000\,000$	-

B.4 Widerstandsreihen

Widerstände, aber auch Kondensatoren und Induktivitäten sind in der Praxis nicht mit jedem beliebigen Bauteilwert erhältlich. Hersteller fertigen diese Bauteile in nominellen Werten nach sogenannten **E-Reihen**. Eine E-Reihe deckt dabei den Wertebereich so ab, dass das Verhältnis aufeinanderfolgender Werte möglichst gleich groß ist. Außerdem sind die Werte so gewählt, dass sich die Ziffernfolgen in jeder Dekade wiederholen. In der Norm DIN IEC 60063 sind sieben E-Reihen definiert: E3, E6, E12, E24, E48, E96 und E192. Dabei gibt die Zahl hinter dem E an, wie viele Werte die Reihe innerhalb einer Dekade enthält. Die E6-Reihe beinhaltet zum Beispiel im Bereich von $1\ \Omega$ bis $1\ \text{M}\Omega$ die folgenden Widerstandsgrößen:

$1.0\ \Omega$, $1.5\ \Omega$, $2.2\ \Omega$, $3.3\ \Omega$, $4.7\ \Omega$, $6.8\ \Omega$, $10\ \Omega$, $15\ \Omega$, $22\ \Omega$, $33\ \Omega$, $47\ \Omega$, $68\ \Omega$, $100\ \Omega$, $150\ \Omega$, $220\ \Omega$, $330\ \Omega$, $470\ \Omega$, $680\ \Omega$, $1.0\ \text{k}\Omega$, $1.5\ \text{k}\Omega$, $2.2\ \text{k}\Omega$, $3.3\ \text{k}\Omega$, $4.7\ \text{k}\Omega$, $6.8\ \text{k}\Omega$, $10\ \text{k}\Omega$, $15\ \text{k}\Omega$, $22\ \text{k}\Omega$, $33\ \text{k}\Omega$, $47\ \text{k}\Omega$, $68\ \text{k}\Omega$, $100\ \text{k}\Omega$, $150\ \text{k}\Omega$, $220\ \text{k}\Omega$, $330\ \text{k}\Omega$, $470\ \text{k}\Omega$, $680\ \text{k}\Omega$, $1\ \text{M}\Omega$

C Arduino® Programmierung - die Basics

Als Hauptkomponente wird während des Praktikums ein Arduino® Uno-kompatibles Mikrocontroller-Board verwendet (siehe Abbildung 7). Der Mikrocontroller wird mittels, auf den Praktikums-PCs vorinstallierter, Arduino-IDE in der Programmiersprache C programmiert. Das Arduino®-Framework bietet dabei einige Vereinfachungen im Gegensatz zu anderen Environments, da es durch vordefinierte Funktionen kaum notwendig ist, direkt Registerwerte des Mikrocontrollers zu setzen. Im folgenden Abschnitt sollen einige wichtige Grundkenntnisse in der Programmierung des verwendeten Mikrocontrollers näher erklärt werden. Zusätzlich bietet das *Foundation-Tutorial* von Arduino® eine gute Übersicht über die wichtigsten Befehle und Einstellungen (www.arduino.cc/en/Tutorial/Foundations).

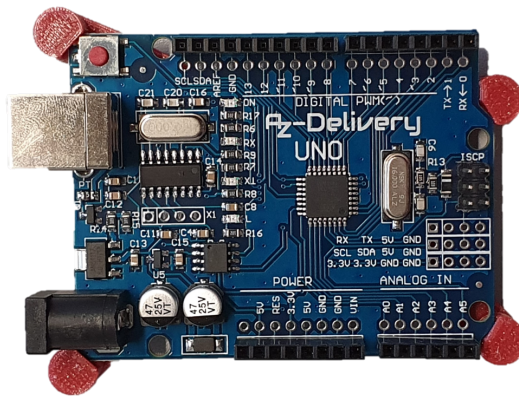


Abbildung 7: Arduino® Uno-kompatibles Mikrocontroller Board

C.1 Arduino® IDE

Zur Programmierung des Mikrocontrollers wird im Praktikum die Entwicklungsumgebung Arduino® IDE verwendet. Diese ist bereits am Praktikums-PC vorinstalliert. Das Softwarepaket kann auch unter www.arduino.cc/en/Main/Software heruntergeladen werden.

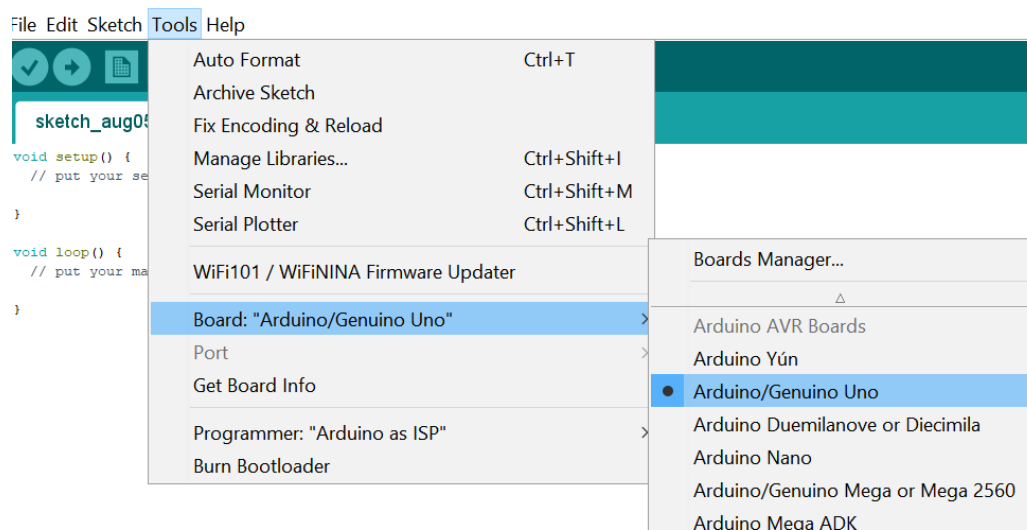


Abbildung 8: Wahl des richtigen Boards

Um den Mikrocontroller zu programmieren muss zunächst unter *Tools* → *Board* das richtige Board ausgewählt werden. Für das Praktikum wird das Board *Arduino/Genuino Uno* benötigt (siehe Abbildung 8).

Weiter muss unter *Tools* → *Port* der richtige COM-Port ausgewählt werden, an dem das Mikrocontroller-Board angeschlossen wurde.

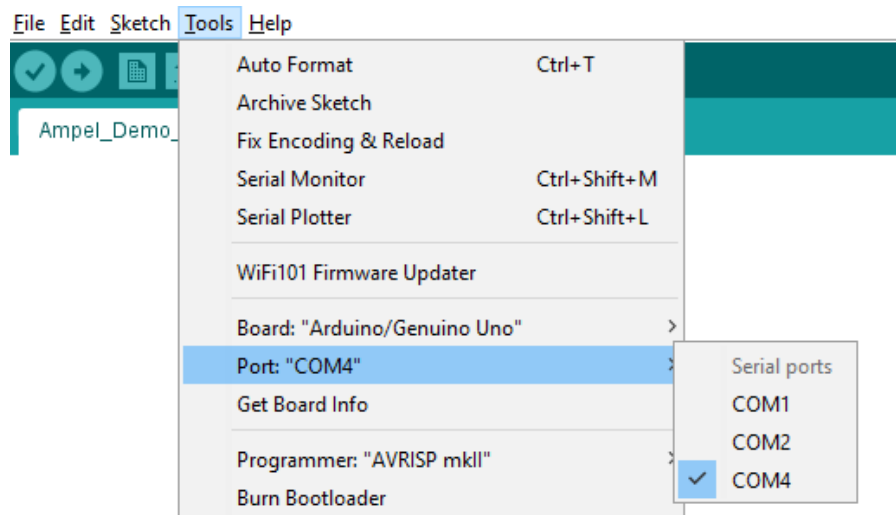


Abbildung 9: Wahl des richtigen COM-Ports

Wurden Board und COM-Port richtig ausgewählt so kann das geschriebene Programm mit einem Klick auf den Pfeil-Button kompiliert und hochgeladen werden.



Abbildung 10: Upload des Programms

C.2 Programmgerüst

Das Programmgerüst eines Arduino®-Sketches besteht aus zwei verschiedenen Funktionen:

- `void setup()`
Diese Funktion wird einmalig beim Start des Programms im Mikrocontroller aufgerufen. Sie wird genutzt um Eingänge und Ausgänge des Mikrocontrollers zu initialisieren und Grundeinstellungen zu definieren.

- `void loop()`

Diese Funktion wird während des Betriebs des Mikrocontrollers dauerhaft wiederholt. Sie wird dazu verwendet um Programm-Befehle abzuarbeiten.

C.3 Grundlegende Befehle und Funktionen

Folgende grundlegende Befehle und Funktionen werden während des Praktikums benötigt:

Befehl	Erklärung
<code>pinMode(pin, mode)</code>	Konfiguriert den angegebenen Pin nach dem angegebenen Modus. Die Variable <code>pin</code> gibt die Nummer des Pins (z.B. 1 bis 13) an. Die Variable <code>mode</code> kann z.B. folgende Werte annehmen: • <code>OUTPUT</code> → Der Pin wird als Ausgang benutzt um z.B. eine LED ein- und auszuschalten • <code>INPUT</code> → Der Pin wird als Eingang benutzt um z.B. Taster einzulesen.
<code>delay(time)</code>	Pausiert das Programm für die Zeit <code>time</code> in Millisekunden
<code>digitalRead(pin)</code>	Liest den Wert eines digitalen Pins (Pin 1 bis Pin 13) und gibt HIGH oder LOW zurück. Die Variable <code>pin</code> gibt die Nummer des Pins an, von dem gelesen werden soll. Der Pin muss zuvor mit dem Befehl <code>pinMode()</code> als Eingang konfiguriert werden.
<code>digitalWrite(pin, value)</code>	Setzt den angegebenen digitalen Pin (Pin 1 bis Pin 13) auf den in <code>value</code> angegebenen Wert. Die Variable <code>value</code> kann folgende Werte annehmen: • <code>HIGH</code> → logisch 1 • <code>LOW</code> → logisch 0 Der Pin muss zuvor mit dem Befehl <code>pinMode()</code> als Ausgang konfiguriert werden.
<code>analogRead(pin)</code>	An den Pins A0 bis A5 können analoge Signale eingelesen werden. Die Pins sind dabei intern mit einem Analog-zu-Digital-Wandler (ADC) verbunden, der die analoge Eingangsspannung in einen digitalen Wert von 0 bis 1023 umwandelt. Ein Wert von 0 entspricht dabei einer Eingangsspannung von 0 V, ein Wert von 1023 entspricht einer Spannung von 5 V. Die Variable <code>pin</code> kann die Werte A0 bis A5 annehmen. Die Funktion gibt den umgewandelten digitalen Wert zurück.
<code>analogWrite(pin, value)</code>	Gibt an dem mit <code>pin</code> angegebenen Mikrocontroller Pin den angegebenen Wert <code>value</code> als Pulsweitenmodulationssignal (PWM-Signal) aus. Folgende Pins können für diese Funktion verwendet werden: Pin 3,9,10 und 11 erzeugen ein PWM-Signal mit einer Frequenz von 490 Hz während Pin 5 und 6 ein PWM-Signal mit einer Frequenz von 960 Hz erzeugen.

C.4 Serial Monitor

Für Debug-Zwecke und zur Ausgabe von Daten am PC, wird das serielle Interface des Mikrocontrollers verwendet. Dieses Interface ist als UART (Universal Asynchronous Receiver Transmitter) implementiert. Mehr Informationen zu UART und serielle Datenübertragung finden Sie unter: de.wikipedia.org/wiki/Universal_Asynchronous_Receiver_Transmitter. Die empfangenen Daten können in der Arduino® IDE am Serial Monitor dargestellt werden. Gleichzeitig können über dieses Interface Daten und Befehle an den Mikrocontroller geschickt werden. Der Serial Monitor kann über das Menü *Tools*→*Serial Monitor* geöffnet werden.

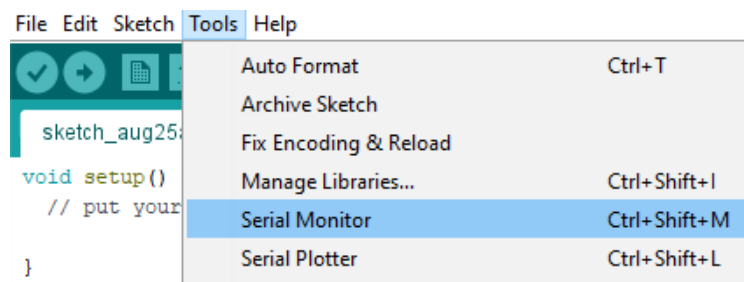


Abbildung 11: Menüpunkt um den Serial Monitor zu öffnen

In Abbildung 12 ist ein Screenshot des geöffneten Serial Monitor ersichtlich. In der Textleiste können Befehle und Daten eingegeben werden, die mithilfe des Sende-Buttons an den Mikrocontroller übertragen werden können. Im Feld darunter werden die empfangenen Daten dargestellt.

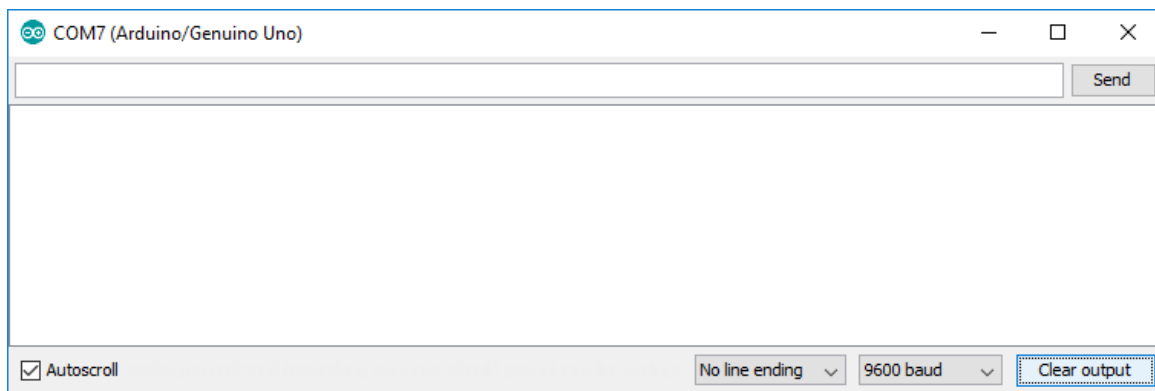


Abbildung 12: Serial Monitor

Unten rechts kann die Baudrate der Übertragung vom PC an den Mikrocontroller eingestellt werden. Diese Rate gibt an, wie viele Bits pro Sekunde gesendet werden. Für eine richtige beidseitige Übertragung zwischen Mikrocontroller und PC (Senden und Empfangen auf beiden Seiten) muss am Serial Monitor die selbe Baudrate eingestellt werden, welche zuvor am Mikrocontroller definiert wurde.

Mit den folgenden Programm-Befehlen kann das Serielle Interface am Mikrocontroller konfiguriert und zum Datenaustausch genutzt werden:

Befehl	Erklärung
<code>Serial.begin(9600)</code>	Wird in der <code>setup()</code> -Funktion verwendet. Öffnet die serielle Schnittstelle und stellt die Datenrate auf 9600 Bit/s ein
<code>Serial.print(val)</code>	Sendet die unter <code>val</code> angegebenen Daten an das serielle Interface. Der Datentyp von <code>val</code> ist dabei beliebig. Jedes zur Übertragung angegebene Zeichen wird als 8 Bit ASCII-Code übertragen. Mehr Informationen zu diesem Befehl unter www.arduino.cc/reference
<code>Serial.println(val)</code>	Sendet die angegebenen Daten an das serielle Interface. Im Unterschied zu <code>Serial.print(val)</code> wird an die unter <code>val</code> angegebenen Daten ein Zeilenumbruch (ASCII-Code) angehängt.
<code>Serial.readString()</code>	Liest die Daten, die mittels des Serial Monitors übergeben werden, ein und gibt diese als String zurück.

C.5 Interrupts

Interrupts sind nützlich um den Mikrocontroller auf spezielle Gegebenheiten, wie z.B. Werteänderungen an Pins, reagieren lassen zu können, ohne ständig die betreffenden Pins abfragen zu müssen. Ein gutes Beispiel ist dafür das Auslesen einer Lichtschranke. Soll auf eine Unterbrechung der Schranke sofort reagiert werden, müsste der Mikrocontroller, ohne Interrupts, dauerhaft den Pin auslesen und kann keine weiteren Berechnungen oder Anweisungen ausführen.

Mittels eines Interrupts kann nun erreicht werden, dass der Mikrocontroller bei einer Unterbrechung der Lichtschranke das laufende Programm unterbricht, in eine spezielle Funktion springt (**I**nterrupt-**S**ervice-**R**outine), diese abarbeitet und danach mit dem laufenden Programm fortfährt. Der verwendete Mikrocontroller stellt 2 Pins zur Verfügung, die mit Interrupts verbunden werden können: Pin 2 und Pin 3.

Mit dem Befehl

```
attachInterrupt(digitalPinToInterrupt(pin), ISR, mode)
```

wird ein Pin mit einem Interrupt verbunden. Die Funktion `digitalPinToInterrupt(pin)` wandelt die angegebene Pin-Nummer (2 oder 3) in eine Interrupt-Nummer um. Der Parameter `ISR` gibt den Namen der Interrupt-Service-Routine (ISR) an, die bei Auftreten eines Interrupts ausgeführt werden soll. Mit `mode` wird definiert, wann der Interrupt getriggert werden soll. Dafür sind 4 Konstanten vordefiniert:

- `LOW` - Interrupt wird getriggert, wenn der Pin LOW ist
- `CHANGE` - Interrupt wird getriggert, wenn der Pin den Wert ändert
- `RISING` - Interrupt wird getriggert, wenn der Pin von LOW auf HIGH wechselt
- `FALLING` - Interrupt wird getriggert, wenn der Pin von HIGH auf LOW wechselt.

Wichtig: Es kann immer nur eine Interrupt-Service-Routine gleichzeitig laufen. Andere Interrupts werden aufgerufen sobald die erste ISR beendet ist. Die Reihenfolge hängt dabei von der Priorität der Routinen ab. Das bedeutet auch, dass Interrupt-Service-Routinen so kurz wie möglich gehalten werden sollten, da während einer Routine nicht sofort auf andere Interrupts reagiert werden kann!

Im folgenden Beispiel-Code wird ein Tastendruck erkannt und mit einer LED angezeigt. Um nicht die gesamte Zeit auf den Tastendruck warten zu müssen, wird ein Interrupt verwendet:

```
1 // Setze den Pin fuer die LED auf 13
2 const byte ledPin = 13;
3 // Setze den Pin fuer den Button auf 2
4 const byte buttonPin = 2;
5 // Definiere eine globale volatile Variable fuer den Status des Button
6 volatile byte state = LOW;
7
8 void setup() {
9   // Lege den Pin fuer die LED als Output fest
10  pinMode(ledPin, OUTPUT);
11  // Lege den Pin fuer den Button als Input fest
12  pinMode(buttonPin, INPUT);
13  // Lege die ISR 'blink' auf den Buttonpin mit Modus 'CHANGE':
14  // "Bei wechselnder Flanke auf dem Buttonpin" --> "Fuehre die ISR aus"
15  attachInterrupt(digitalPinToInterrupt(buttonPin), blink, CHANGE);
16 }
17
18 void loop() {
19   // Platz fuer andere Berechnungen
20 }
21
22 void blink() {
23   state = digitalRead(buttonPin);
24   digitalWrite(ledPin, state);
25 }
```

Interrupts werden auch vom Mikrocontroller intern genutzt. Die Funktion `delay()` benutzt beispielsweise Interrupts. Daher kann diese Funktion in einer ISR nicht verwendet werden.

Weitere Informationen zu diesem Themengebiet finden Sie unter www.arduino.cc/reference.

C.6 Verwendung von Tastern

In vielen populären Anwendungen von Mikrocontrollern soll der Controller auf Benutzereingaben reagieren und dementsprechend Programmteile ausführen. Die simpelste Form der Benutzereingabe stellt dabei ein einfacher Taster dar. Mit ihm können zum Beispiel digitale Werte von 1 und 0 an Eingangspins des Mikrocontrollers gelegt werden. Dabei muss allerdings auf die Verschaltung des Tasters mit dem Mikrocontroller-Pin geachtet werden.

Würde man einfach nur den Taster zwischen den Mikrocontroller-Pin und der Versorgungsspannung oder auch Masse (GND) hängen (wie in den Abbildungen 13a und 13b ersichtlich) würde im Falle eines nicht gedrückten Schalters kein definierter Zustand (also weder 1 noch 0) am Eingangspin für den Mikrocontroller sichtbar sein. Das bedeutet, dass nicht sauber zwischen 0 und 1 mithilfe des Tasters geschaltet werden kann. Um dieses Problem zu umgehen, kann der Eingangspin in dem Falle, dass der Schalter gerade nicht gedrückt wird, fix mit dem jeweils anderen Potential verbunden werden (siehe Abbildungen 13c und 13d). Wird also der Schalter gegen Masse gehängt, so könnte man den Mikrocontroller-Pin doch fix mit der Versorgungsspannung verbinden. Dieser Ansatz bringt jedoch das Problem mit sich, dass im Falle eines Tasterdrucks ein Kurzschluss entsteht. Während des Kurzschlusses

fließen große Ströme, die den Mikrocontroller auch zerstören können. Die richtige Verschaltung eines Tasters an einem Mikrocontroller Pin ist in den Abbildungen 13e und 13f dargestellt. Dafür wird jeweils ein Widerstand mit einem Wert von $\geq 10\text{ k}\Omega$ benötigt, welchen den maximal fließenden Strom auf 0.5 mA begrenzt.

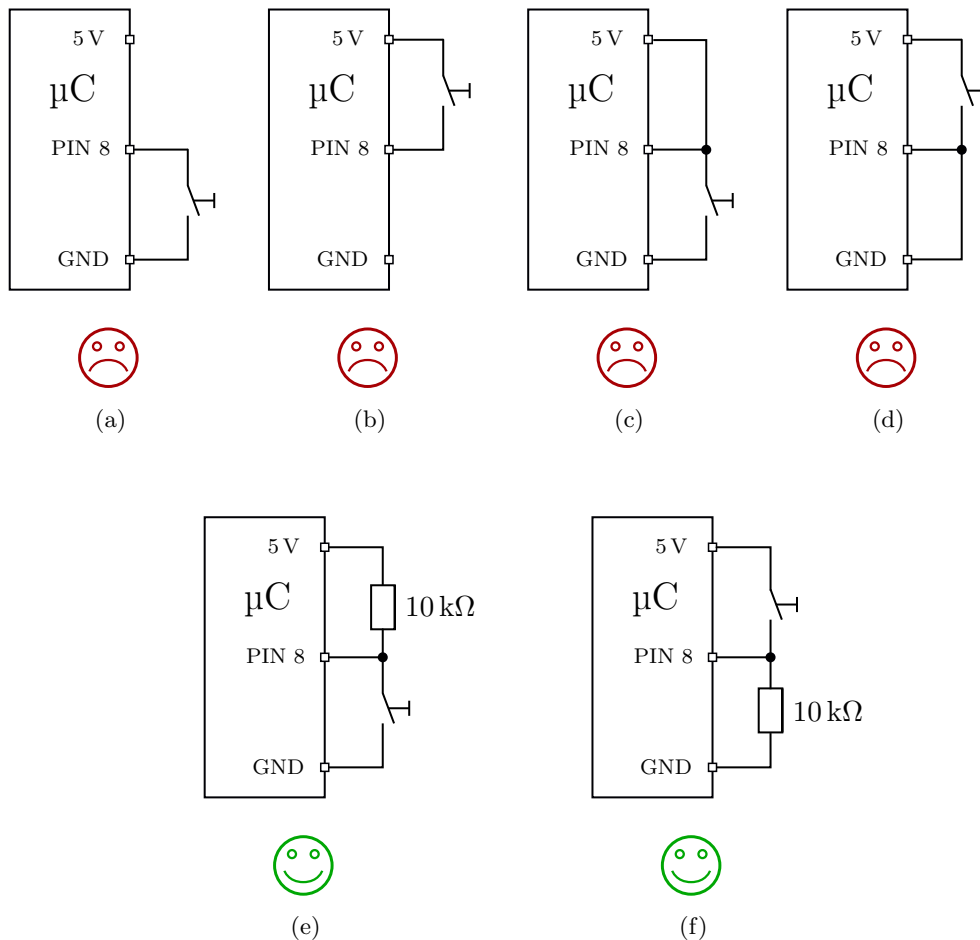


Abbildung 13: Falsche und Richtige Vorgangsweise zur Verschaltung von Tastern mit dem Mikrocontroller

Für den Fall, dass der Taster gegen Masse geschaltet werden soll (ein Tastendruck soll also eine logische 0 am Eingang des Mikrocontrollers verursachen) wird der Widerstand zwischen Eingangspin und Versorgungsspannung gehängt. Dieser Widerstand schützt den Mikrocontroller vor einer Zerstörung durch zu hohe Ströme und „zieht“ gleichzeitig die anliegende Spannung nach oben in Richtung Versorgungsspannung $\Rightarrow$ **Pull-UP Widerstand**

Dasselbe Prinzip gilt für den Fall, dass der Taster gegen Versorgungsspannung geschaltet ist (das heißt bei Tastendruck liegt am Mikrocontroller-Pin eine logische 1 an). Der Widerstand schützt wiederum den Mikrocontroller und „zieht“ gleichzeitig das Potential am Eingangspin gegen Masse $\Rightarrow$ **Pull-DOWN Widerstand**

Am verwendeten Mikrocontroller sind bereits Pull-UP Widerstände für die digitalen Pins vorgesehen. Um einen Pin mit einem dieser Widerstände zu verwenden muss folgender Pin-Mode zu Beginn des Mikrocontroller-Programms konfiguriert werden:

```
pinMode(pin, INPUT_PULLUP)
```

C.7 Hilfe während des Praktikums

Die nachfolgenden Aufgaben sollen an den jeweiligen Praktikumstagen im PC Labor des Institutes für Integrierte Schaltungen durchgeführt werden. Sie können auch gerne das Internet zur Hilfe nehmen. Alle Funktionen und Befehle zur Programmierung des Mikrocontrollers können jedoch auch offline über die Arduino® IDE nachgeschlagen werden. Wählen Sie dazu in der IDE dem Button *Help* ⇒ *Reference* (Siehe Abbildung 14).

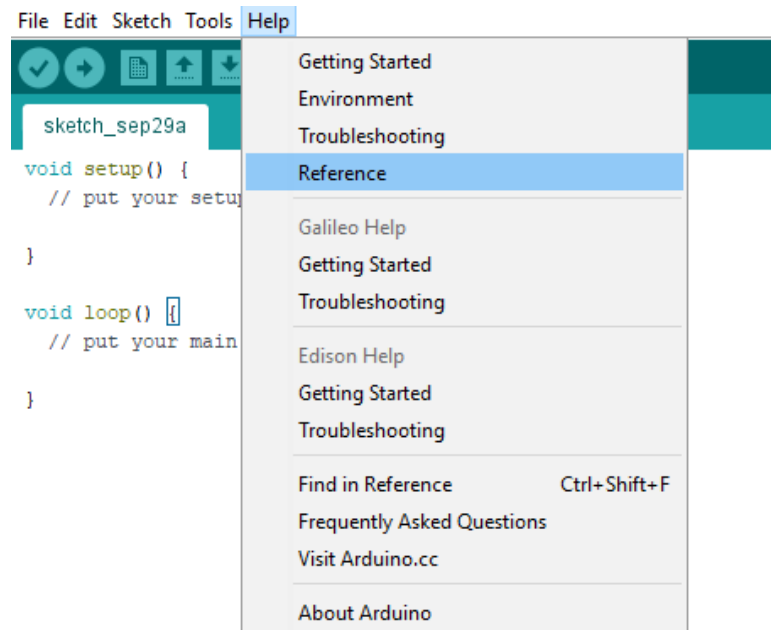


Abbildung 14: Menüpunkt um die Arduino®-Referenz zu öffnen

Es öffnet sich die offline Language-Referenz in der jede, der oben beschriebenen Funktionen (und noch viele weitere) mit Beispielen erläutert wird.

1 Ampelsteuerung

In dieser Aufgabe soll der Ablauf einer einfachen Ampel für den Straßenverkehr mit Hilfe des Mikrocontrollers simuliert werden. Die Ampel wird dabei durch 3 verschiedenfarbige Leuchtdioden (LEDs) (rot, gelb und grün) am Steckbrett visualisiert. Der Ablauf einer Ampelsteuerung (Autoampel) setzt sich aus 4 verschiedenen Phasen zusammen:

- Phase 1:
Die Ampel steht auf rot. Diese Phase soll im Praktikum 15s andauern. In der Realität ist diese Zeitspanne von der Gegebenheit der Ampelanlage, sowie der Verkehrslage abhängig.
- Phase 2:
Nach Abschluss der ersten Phase, leuchtet für die Dauer von 2s die gelbe Leuchte der Ampel gemeinsam mit der roten. Danach folgt Phase 3.
- Phase 3:
Diese Phase stellt die sogenannte Grünphase dar. Im Praktikum soll die Ampel für insgesamt 15s grün leuchten. Die dritte Phase wird durch viermaliges Blinken der Grünen Leuchte beendet. Dabei entsprechen die Zeiten der Leucht- und Dunkelphasen jeweils 500 ms.
- Phase 4:
Nach dem Ende der Grünphase wird wieder über eine kurze Gelbphase (Phase 4) zu Rot (Phase 1) gewechselt. Die Gelbe Leuchte soll dabei 2s lang leuchten.



Nach Beendigung der vierten Phase wird der Kreislauf wieder von vorne, bei Phase 1 startend, begonnen.

1.1 Vorbereitung

Die Ausgangsspannung an den digitalen Pins des Mikrocontrollers beträgt $V_{\text{out}} = 5\text{ V}$. Die benötigte Diodenspannung der LEDs beträgt laut Datenblatt ca. $U_D = 2\text{ V}$. Um die LEDs nicht zu zerstören soll durch einen Vorwiderstand der Strom durch die Diode auf $I_D = 15\text{ mA}$ begrenzt werden.

Aufgabe:

Berechnen Sie den benötigten Vorwiderstand R_V und wählen Sie einen geeigneten Widerstand aus der E6 Reihe als Bauteil aus:

Berechnet:

$$R_{V,\text{calc}} = \underline{\hspace{2cm}} \Omega$$

Aus E6-Reihe gewählt:

$$R_V = \underline{\hspace{2cm}} \Omega$$

1.2 Praktikumsaufgabe

- a) Bauen Sie aus 3 verschiedenfarbigen LEDs (grün, gelb und rot) eine Autoampel am Steckbrett auf. Benutzen Sie als Vorwiderstand zu den LEDs jeweils einen Widerstand R_V . Testen Sie mit Hilfe der konstanten Spannungsversorgung (5 V-Pin, siehe Abbildung 15) des Mikrocontroller-Boards ob alle 3 LEDs leuchten.

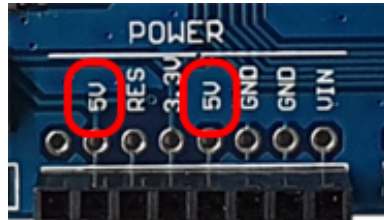


Abbildung 15: Konstante 5 V-Spannungsquelle am Mikrocontroller-Board

- b) Verbinden Sie die drei LEDs (gemeinsam mit ihrem Vorwiderstand), anstatt der konstanten Spannungsquelle, mit beliebigen digitalen Pins am Mikrocontroller-Board. Die Pins können Sie selbst wählen.
- c) Implementieren Sie den Ablauf der oben angegebenen Ampelsteuerung am Mikrocontroller. Sie können für die Wartezeiten zwischen und während den Phasen durch die Funktion `delay(time)` verwenden.
- d) Zeigen Sie Ihre fertige Schaltung und den implementierten Code dem Praktikumsleiter/Tutor bzw. der Praktikumsleiterin/Tutorin.

1.3 Checkliste für das Protokoll

- ☐ Dokumentieren Sie die Schritte zur Berechnung des Vorwiderstandes R_V
- ☐ Welchen Widerstand haben Sie für die weiterfolgende Implementierung am Steckbrett gewählt und warum?
- ☐ Welche Pins haben Sie für die LEDs am Mikrocontroller gewählt?
- ☐ Wie wurden die Pins konfiguriert?
- ☐ Dokumentieren Sie im Protokoll auch Ihren Code. Stellen Sie sicher, dass der Code über ausreichend viele und aussagekräftige Kommentare verfügt um Ihre Gedankengänge während des Programmierens nachvollziehen zu können

2 Reaktionsspiel

In dieser Aufgabe ist ein Reaktionsspiel bestehend aus 5 LEDs und einem Taster zu realisieren. Die verwendeten LEDs (2×rot, 2×gelb, 1×grün) sollen, wie in Abbildung 16 ersichtlich, am Steckbrett platziert werden.

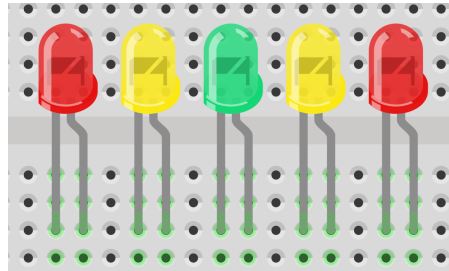


Abbildung 16: Vorgegebene Reihenfolge der LEDs am Steckbrett

Spielablauf:

Die LEDs werden nacheinander als Laufflicht ein- und ausgeschaltet. Leuchtet die mittlere LED auf, so muss der Taster gedrückt werden. Reagiert der Spieler rechtzeitig und drückt den Taster während die LED grün leuchtet, so hat er das Level erfolgreich beendet und steigt zum nächsten Level auf. Im nächsten Level läuft das Laufflicht schneller. Das heißt die Zeit zwischen dem Aufleuchten zweier benachbarter Dioden verringert sich.

Wird der Taster zum falschen Zeitpunkt gedrückt, blinken alle LEDs gleichzeitig dreimal auf und das Spiel startet von vorne (mit dem langsamsten Level).

2.1 Vorbereitung

Die Ausgangsspannung an den digitalen Pins des Mikrocontrollers beträgt $V_{\text{out}} = 5\text{ V}$. Die benötigte Diodenspannung der LEDs beträgt laut Datenblatt ca. $U_{\text{D}} = 2\text{ V}$. Um die LEDs nicht zu zerstören soll durch einen Vorwiderstand der Strom durch die Diode auf $I_{\text{D}} = 15\text{ mA}$ begrenzt werden.

Aufgabe 1:

Berechnen Sie den benötigten Vorwiderstand R_{V} und wählen Sie einen geeigneten Widerstand aus der E6 Reihe als Bauteil aus:

Berechnet:

$$R_{\text{V,calc}} = \underline{\hspace{2cm}} \Omega$$

Aus E6-Reihe gewählt:

$$R_{\text{V}} = \underline{\hspace{2cm}} \Omega$$

Aufgabe 2:

Wie kann das Mikrocontroller-Programm zu einem beliebigen Zeitpunkt gestoppt werden? Wie kann ausgewertet werden, ob der Taster zum richtigen oder zum falschen Zeitpunkt gedrückt wurde?

2.2 Praktikumsaufgabe

- a) Stecken Sie die LEDs wie in Abbildung 16 am Steckbrett auf. Verbinden Sie die LEDs mit den dazugehörigen Vorwiderständen $R_{V,x}$ (entnehmen Sie die entsprechenden Widerstandswerte der Vorbereitung). Testen Sie mit Hilfe der konstanten Spannungsversorgung (analog zu Aufgabe 1) ob alle 5 LEDs leuchten.
- b) Schließen Sie die LEDs an geeignete Pins des Mikrocontrollers an. Konfigurieren Sie in der Funktion `setup()` die verwendeten Pins richtig.
- c) Programmieren Sie ein Lauflicht (von links nach rechts), das alle 5 s zur nächsten LED weiter springt.
- d) Platzieren Sie den Taster am Steckbrett. Verbinden Sie den Taster mit einem geeigneten Pin am Mikrocontroller. Konfigurieren Sie diesen Pin in der Funktion `setup()` dementsprechend.
- e) Fügen Sie dem bereits implementierten Lauflicht folgende Funktionalität hinzu: Bei Tastendruck soll das Lauflicht stehen bleiben. Wird der Taster losgelassen so läuft das Lauflicht weiter (mit gleichbleibender Geschwindigkeit).
- f) Implementieren Sie nun die unter **Spielablauf** angegebene Funktionalität des Reaktionsspiels. Starten Sie mit einer Lauflicht-Geschwindigkeit von 5 s. Jedes mal wenn der Spieler das Level gewinnt, soll die Geschwindigkeit verdoppelt werden!
- g) Testen Sie das Reaktionsspiel auf einwandfreie Funktionalität (Taster wurde zum richtigen Zeitpunkt gedrückt, Taster wurde zum falschen Zeitpunkt gedrückt, ect.).
- h) Zeigen Sie Ihre fertige Schaltung und den implementierten Code dem Praktikumsleiter/Tutor bzw. der Praktikumsleiterin/Tutorin.

2.3 Checkliste für das Protokoll

- ☐ Dokumentieren Sie die Schritte zur Berechnung des Vorwiderstandes R_V
- ☐ Dokumentieren Sie Ihre Überlegungen zu Aufgabe 2 der Vorbereitung
- ☐ Welchen Widerstand haben Sie für die weiterfolgende Implementierung am Steckbrett gewählt und warum?
- ☐ Welche Pins haben Sie für die LEDs am Mikrocontroller gewählt?
- ☐ Wie wurden die Pins konfiguriert?
- ☐ Welchen Pin haben Sie für den Taster gewählt?
- ☐ Wie wurde dieser Pin konfiguriert?
- ☐ Dokumentieren Sie die einzelnen Aufgaben aus Abschnitt 2.2. Legen Sie Ihren Programmcode dem Protokoll bei und erklären Sie die durchgeführten Schritte.
- ☐ Welche Ergebnisse konnten Sie bei dem Test des Reaktionsspiels beobachtet?

3 Realisierung eines Logic Analyzers

Logische Gatter sind elektronische Schaltungen und werden zur Realisierung von booleschen Funktionen verwendet, welche binäre Eingangssignale zu einem binären Ausgangssignal verarbeiten. Die logischen „Grundfunktionen“, welche in digitalen Schaltungen eingesetzt werden sind **NOT**, **AND**, **OR**, **NAND**, **NOR** und **XOR**. Heutzutage werden diese Grundgatter in digitalen Schaltungen in millionenfacher Anzahl pro Chip (IC = Integrated Circuit) eingesetzt. Dabei werden die einzelnen Gatter nicht manuell vom Digital-Designer platziert, sondern durch Synthese einer Hardware-Beschreibungssprache (z.B. VHDL) erzeugt. In der nachfolgenden Praktikumsaufgabe werden Logik-ICs verwendet, die nur eine Art von Grundgattern (in mehrfacher Ausführung) beinhalten. Solche Logik-Bausteine wurden historisch verwendet, um einfache logische Funktionen auf einer Platine zu implementieren.

3.1 Vorbereitung

In Wahrheitstabellen können die möglichen Ausgangszustände eines Logikgatters in Abhängigkeit von den Eingangszuständen dargestellt werden. In einer solchen Tabelle, werden alle möglichen Kombinationen der Eingangssignale gemeinsam mit den entsprechenden Ausgangssignalen aufgelistet.

Aufgabe:

Vervollständigen Sie die unten stehenden Wahrheitstabellen für die Logigatter AND, OR, NAND, NOR und XOR. Die angegebenen Gatter verknüpfen die beiden Eingänge A und B zu einem Ausgang Y. Geben Sie zusätzlich die boolesche Funktion als Formel an.

AND		
A	B	Y
0	0	
0	1	
1	0	
1	1	

Y = _____

OR		
A	B	Y
0	0	
0	1	
1	0	
1	1	

Y = _____

NAND		
A	B	Y
0	0	
0	1	
1	0	
1	1	

Y = _____

NOR		
A	B	Y
0	0	
0	1	
1	0	
1	1	

Y = _____

XOR		
A	B	Y
0	0	
0	1	
1	0	
1	1	

Y = _____

3.2 Praktikumsaufgabe

Mit Hilfe des Mikrocontrollers soll ein Logic-Analyzer realisiert werden, der die logische Funktion eines Logik ICs anhand seiner Eingangs- und Ausgangscharakteristik erkennt. Im Praktikum werden dafür Logikbausteine der Familie SN74HC der Firma Texas Instruments verwendet. Diese Bauteile verfügen jeweils über 14 Pins und beinhalten jeweils 4 Stück der implementierten Logikgatter. In Abbildung 17 ist der Aufbau eines solchen Logikbausteines ersichtlich. Um den Aufbau am Steckbrett zu vereinfachen, soll in den folgenden Praktikumsaufgaben immer das erste (blau eingezeichnete Gatter) bestimmt werden!

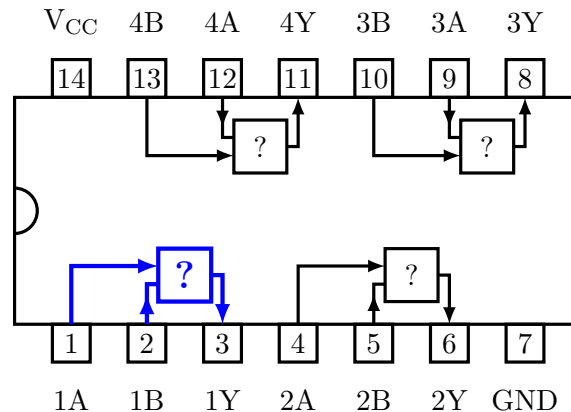


Abbildung 17: Aufbau eines Logikgatters der Bauteilfamilie SN74HC

Zur Inbetriebnahme des Logikbausteins müssen außer den 3 Pins des unbekannten Gatters auch die beiden Pins V_{CC} und GND angeschlossen werden. GND wird mit dem Ground Pin (ebenfalls mit GND beschriftet) des Mikrocontrollerboards verbunden. Der Pin V_{CC} wird mit dem 5 V-Ausgang des Mikrocontrollers verbunden.

- Bereiten Sie am Steckbrett einen *Teststeckplatz* für drei unbekannte Logikbausteine, den DUTs (**D**evice **U**nder **T**est) vor. Achten Sie dabei auf die richtige Verdrahtung der ICs. Die Kerbe (siehe Abbildung 17, links) gibt an, auf welcher Seite (links oder rechts) der Pin 1 zu finden ist.
- Konfigurieren Sie nun am Mikrocontroller 2 digitale Pins als Ausgang und einen digitalen Pin als Eingang. Verbinden Sie die 3 Pins mit den zugehörigen Anschlüssen des DUTs am Steckbrett. A und B (Pin 1 und 2 des DUTs) sind die Eingänge des unbekannten Logikgatters. Y (Pin 3) ist der Ausgang des DUTs.
- Erstellen Sie nun ein Programm, mithilfe dessen Sie mit dem Mikrocontroller die logische Funktions des jeweiligen DUTs feststellen können. Dabei soll die Wahrheitstabelle des unbekannten Logikgatters abgeprüft und daraus ermittelt werden, welche boolesche Funktion der DUT realisiert. Das Ergebnis dieses Tests soll über die serielle Schnittstelle am PC ausgegeben werden.
- Überprüfen Sie mit Ihrem Programm die gegebenen unbekannten Logikgatter DUT 1, DUT 2 und DUT 3. Welche Logikfunktion steckt hinter welchem IC?
- Überprüfen Sie die Wahrheitstabelle der gegebenen Gatter händische mithilfe des Multimeters. Stimmt diese Auswertung mit dem Ergebnis des Mikrocontroller-Programms überein?
- Zeigen Sie Ihre fertige Schaltung und den implementierten Code dem Praktikumsleiter/Tutor bzw. der Praktikumsleiterin/Tutorin.

3.3 Checkliste für das Protokoll

- ☐ Dokumentieren Sie Ihre Ergebnisse aus der Vorbereitung
- ☐ Welche Pins wurden für die Verbindung zum jeweiligen DUT verwendet?
- ☐ Dokumentieren Sie Ihre Überlegungen zur Programmierung des Logic-Analyzers
- ☐ Wie wurde die Überprüfung der logischen Funktion implementiert? Auf welche logischen Grundfunktionen wurde dabei der jeweilige DUT überprüft?
- ☐ Wie bewerten Sie die Flexibilität und Performance Ihrer Implementierung? Ist Ihr Code einfach auf weitere logische Funktionen zu erweitern?
- ☐ Welche logischen Funktionen konnten Sie den angegebenen DUTs mithilfe Ihres Mikrocontroller-Programms zuordnen?
- ☐ Dokumentieren Sie die Ergebnisse der händischen Überprüfung der gegebenen DUTs. Stimmen diese Ergebnisse mit Ihrem den Resultaten Ihres Programms überein?

4 Wertebestimmung von Widerstand und Kondensator

Widerstände und Kondensatoren sind zwei grundlegende Bauteile der Elektrotechnik. Beide dieser Bauelemente haben Sie bereits in Elektronik kennengelernt. In der folgenden Praktikumsaufgabe sollen die Werte dieser beiden Bauteile mit dem Mikrocontroller bestimmt werden.

4.1 Vorbereitung

Im Praktikum werden Widerstände in einer bedrahteten Ausführung verwendet. Auf den Gehäusen der einzelnen Widerstände sind farbige Ringe gedruckt. Diese geben, wie in Kapitel B.3 beschrieben, den elektrischen Wert des Bauteils inklusive Toleranz an.

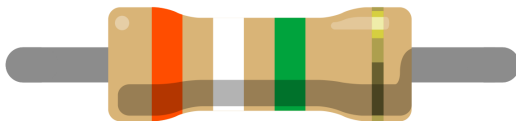
Aufgabe 1:

Geben Sie für die folgenden abgebildeten Widerstände den Wert in Ω an. Bestimmen Sie weiters die Toleranz der einzelnen Bauteile.



Widerstandswert: _____

Toleranz: _____



Widerstandswert: _____

Toleranz: _____



Widerstandswert: _____

Toleranz: _____

Aufgabe 2:

An einem Spannungsteiler (siehe Abbildung 18) liegt eine Eingangsspannung U_e von 5 V an. Die Ausgangsspannung U_a wird zu 3 V gemessen.

- Wie muss das Verhältnis der Widerstandswerte beider Widerstände gewählt werden, damit dieser Zustand erreicht wird? Wie groß muss der Widerstand R_2 sein, wenn für den Widerstand $R_1 = 10 \text{ k}\Omega$ gilt?
- Sie tauschen den Widerstand R_2 gegen einen unbekannten Widerstand R_x aus. R_1 bleibt weiterhin $10 \text{ k}\Omega$. Die Ausgangsspannung wird wieder gemessen und beträgt $U_a = 1 \text{ V}$. Wie groß ist der Widerstand R_x ?

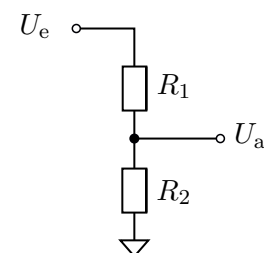


Abbildung 18: Spannungsteiler

a) Berechnung von R_1/R_2 und R_2 :

$$\frac{R_1}{R_2} = \underline{\hspace{2cm}}$$

$$R_2 = \underline{\hspace{2cm}}$$

b) Berechnung von R_x :

$$R_x = \underline{\hspace{2cm}}$$

Aufgabe 3:

Ein Kondensator wird über einen Widerstand $R = 10\text{ k}\Omega$ an einer Spannungsquelle U_q geladen. Zu Beginn des Aufladevorgangs beträgt die Spannung am Kondensator 0 V . Nach 100 ms kann die Spannung am Kondensator zu 63% der Spannung U_q gemessen werden.

- a) Wie groß ist die Kapazität des Kondensators?
- b) Wie lange dauert es, bis der Kondensator vollständig ($> 99\%$) aufgeladen ist?

a) Berechnung von C :

$$C = \underline{\hspace{2cm}}$$

b) Berechnung von $t_{99\%}$

$$t_{99\%} = \underline{\hspace{2cm}}$$

4.2 Praktikumsaufgabe

4.2.1 Bestimmung von Widerständen mit dem Mikrocontroller

Im Praktikums-Baukasten finden Sie ein Säckchen mit 4 unterschiedlichen unbekannten Widerständen (Beschriftung: ? Ohm). Mit dem Mikrocontroller soll nun der elektrische Wert dieser unbekannten Bauteile ermittelt werden. Der unbekannte Widerstand wird dabei mit einem Widerstand $R = 10\text{ k}\Omega$ zu einem Spannungsteiler verschalten. Die Eingangsspannung U_e sind dabei die 5 V des Mikrocontrollerboards. Führen Sie nun folgende Aufgabenschritte durch:

- Verwenden Sie nun den analogen Eingang des Mikrocontrollers, um die Ausgangsspannung am Spannungsteiler zu messen.
- Berechnen Sie mithilfe des Mikrocontrollers den Widerstandswert des unbekannten Widerstandes und geben Sie diesen Wert über den seriellen Monitor am PC aus.
- Überprüfen Sie Ihr Programm für jeden unbekannten Widerstand mit den folgenden Mitteln:
 - Ablesen des Widerstandswertes mithilfe des Farbcodes
 - Messung des Widerstandes mit dem Ohmmeter
 - Messung des Widerstands durch Strom- und Spannungsmessung mit dem Multimeter (Ohmsches Gesetz!)
- Zeigen Sie Ihre fertige Schaltung und den implementierten Code dem Praktikumsleiter/Tutor bzw. der Praktikumsleiterin/Tutorin.

4.2.2 Bestimmung von Kondensatoren mit dem Mikrocontroller

Da Ihr Multimeter nicht über die Möglichkeit verfügt, den genauen Kapazitätswert eines Kondensators zu bestimmen, entscheiden Sie sich, selbst ein Kapazitäts-Messgerät mithilfe des Mikrocontrollers zu bauen. Verwenden Sie dazu das Prinzip der RC-Ladekurve und führen Sie die folgenden Aufgabenschritte durch:

- Wählen Sie den Widerstand, über den der Kondensator geladen werden soll zu $10\text{ k}\Omega$.
- Überlegen Sie, wie durch Messung von Spannungswerten am analogen Eingang des Mikrocontrollers der Kapazitätswert bestimmt werden kann.
- Um ein genaues Messergebnis erzielen zu können, muss sichergestellt werden, dass der unbekannte Kondensator zu Beginn der Messung immer vollständig entladen/geladen sein muss. Überlegen Sie, wie dies mithilfe des Mikrocontrollers automatisiert durchgeführt werden kann.
- Berechnen Sie im Mikrocontrollerprogramm den Wert des Kondensators und geben Sie das Ergebnis am seriellen Monitor aus.
- Überprüfen Sie Ihr Programm mit den im Baukasten vorhandenen Kondensatoren ($47\text{ }\mu\text{F}$, $100\text{ }\mu\text{F}$ und $220\text{ }\mu\text{F}$). Da es sich um Elektrolyt-Kondensatoren handelt, achten Sie auf die richtige Polarität! Bei falscher Polung kann der Kondensator zerstört werden.
- Zeigen Sie Ihre fertige Schaltung und den implementierten Code dem Praktikumsleiter/Tutor bzw. der Praktikumsleiterin/Tutorin.

4.3 Checkliste für das Protokoll

- ☐ Dokumentieren Sie Ihre Ergebnisse aus der Vorbereitung
- ☐ Erklären Sie Ihre Überlegungen zur Messung von Widerstandswerten mit dem Mikrocontroller
- ☐ Dokumentieren Sie die Messergebnisse zu den unbekannten Widerständen, sowie die Ergebnisse des Mikrocontrollerprogramms. Können Sie Unterschiede in den Messergebnissen feststellen? Wenn ja, wodurch könnten diese verursacht werden?
- ☐ Erklären Sie das angewandte Messprinzip. Welche Überlegungen haben Sie dazu angestellt?
- ☐ Dokumentieren Sie die Ergebnisse des Mikrocontrollerprogramms bei der Messung von bekannten Kondensatoren. Können Sie Abweichungen von den angegebenen Werten feststellen?

5 Ansteuerung eines Lautsprechers

In dieser Praktikumsaufgabe sollen mithilfe des Mikrocontrollers Töne an einem Lautsprecher ausgegeben werden. Dafür soll an Ausgangspins des Mikrocontrollers ein 4 Bit großer digitaler Wert ausgegeben werden. Dieser digitale Wert soll anschließend durch eine geeignete Schaltung am Steckbrett in ein analoges Signal gewandelt und an den Lautsprecher angelegt werden.

Mikrocontroller Register

Um mehrere Ausgänge des Mikrocontrollers gleichzeitig und mithilfe eines einzigen Befehls anzusteuern, können die sogenannten Port-Register des Mikrocontrollers direkt manipuliert werden. Die IO-Pins des Mikrocontrollers sind in der Hardware jeweils mit Registern hinterlegt. Dabei werden mehrere Pins zu einem Port zusammengefasst (Bei 8 Bit-Mikrocontrollern bilden immer 8 Pins einen Port). Für jeden Port des Mikrocontrollers stehen in Hardware 3 unterschiedliche Register zur Verfügung:

- **DDR:**
Mit diesem Datenrichtungsregister wird angegeben ob ein Pin als Eingang oder als Ausgang definiert werden soll.
- **PORT:**
Dieses Register setzt einen als Ausgang definierten Pin auf **HIGH** oder **LOW**.
- **PIN:**
Das PIN Register liest das anliegende Spannungslevel am Pin ein (**HIGH** oder **LOW**).

Bisher wurde zum Setzen der Pinrichtung, sowie zum Lesen und Schreiben auf Pins die Funktionen der Arduino-Bibliothek verwendet (`pinMode(pin, mode)`, `digitalRead(pin)` und `digitalWrite(pin, value)`). Diese können eins zu eins auf Operationen der drei Mikrocontroller-Register umgelegt werden. So schreibt die Methode `pinMode(pin, mode)` an die durch `pin` angegebenen Bit-Stelle im DDR-Register eine 0 oder 1, je nach angegebenen `mode`. Gleiches gilt für die beiden Schreib- und Lese-Funktionen. `digitalWrite(pin, value)` schreibt an die entsprechende Stelle im PORT-Register den in `value` angegebenen Wert. `digitalRead(pin)` liest das im PIN-Register an der Stelle `pin` gespeicherte Bit aus.

Die drei Register können jedoch nicht nur über die Bibliotheks-Funktionen beschrieben und ausgelesen werden, sondern auch durch direkte Zuweisung im Code. Beispielsweise können alle Pins des Port D eines Mikrocontrollers mit dem folgenden Befehl als Ausgang konfiguriert werden. In der zweiten Zeile des nachfolgenden Codes werden alle zuvor konfigurierten Ausgangspins gleichzeitig auf 1 also logisch **HIGH** gesetzt.

```
1 DDRD = 0xFF;  
2 PORTD = 0xFF;
```

Eine direkte Zuweisung mit `=` auf das Register muss nicht immer sinnvoll sein. Sollen beispielsweise einige der Pins in ihrem vorherigen Zustand belassen werden, so würde man mit einer direkten Zuweisung diese Zustände überschreiben. Eine einfache Art dies zu verhindern ist die Anwendung von logischen Operationen zur Zuweisung von neuen Pin-Zuständen. Sollen zum Beispiel gewisse Pins auf logisch **HIGH** gesetzt werden, so bietet sich eine OR-Verknüpfung mit einer Maske an, die an den Stellen, an denen die Pins auf **HIGH** gesetzt werden 1er stehen hat:

```
1 PORTD |= 0x01;
```

In diesem Beispiel wird nur der Pin 0 auf **HIGH** gesetzt. Alle anderen bleiben auf ihrem ursprünglichen Wert. Mit der Verknüpfung `&=` können selektiv Bits bzw. Pins eines Ports auf **LOW** gesetzt werden. Dabei wird, wie schon in ähnlicher Art zuvor, eine Maske verknüpft, die an den gewünschten Stellen 0en aufweist:

```
1 PORTD &= 0xFE;
```

In dem oben angeführten Beispiel wird nur das erste Bit des Ports D auf 0 gesetzt. Die anderen Bits bzw. Pins behalten ihren vorherigen Zustand.

Der im Praktikum verwendete Mikrocontroller ATmega328P verfügt insgesamt über drei 8 Bit-Ports (Port B, C und D). In Abbildung 19 ist die Pinbelegung des Mikrocontrollers ersichtlich. Port D entspricht dabei den digitalen Pins 0 bis 7. Port B entspricht den Pins 8 bis 13. Die beiden höchsten Pins des Ports sind mit dem Crystal-Quarz am Mikrocontroller-Board verbunden und können (bzw. sollten) daher nicht benutzt werden. Port C wird den analogen Pins 0 bis 5 zugeordnet. Pin 6 und Pin 7 können nicht direkt angesprochen werden (C6 entspricht zum Beispiel dem Reset-Pin des Mikrocontrollers).

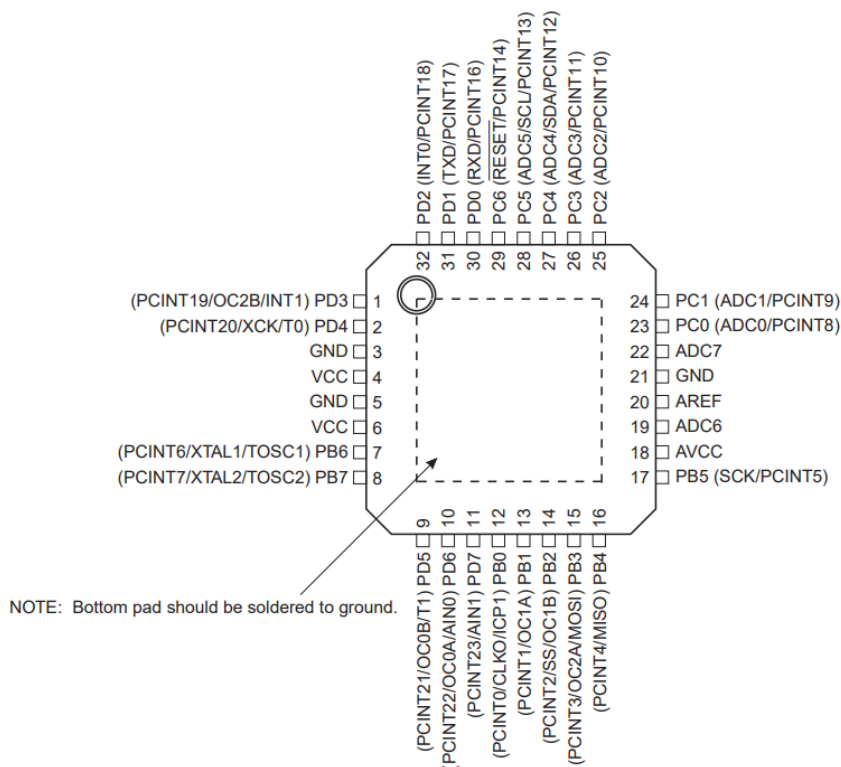


Abbildung 19: Pin-Konfiguration des ATmega328P

Wandlung von Digital zu Analog am Steckbrett

Sogenannte Digital-to-Analog Converter (DAC) werden in der Schaltungstechnik dazu verwendet, um digitale Signale in analoge Signale umzuwandeln. In dieser Praktikumsaufgabe soll eine sehr einfache Realisierung eines DAC am Steckbrett aufgebaut werden: ein **R/2R Netzwerk**.

Dieses Netzwerk ist aus Widerständen mit den Werten R und $2R$, wie in Abbildung 20 ersichtlich, aufgebaut.

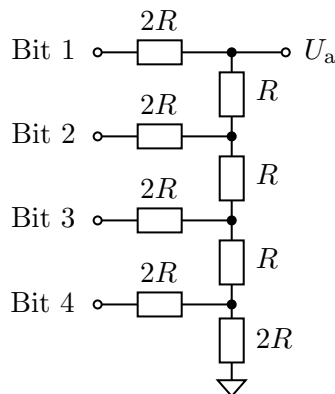


Abbildung 20: R2R Netzwerk für 4 digitale Bits

An den einzelnen Eingangsbits der Schaltung (Bit 1 bis Bit 4) liegt entweder eine Spannung von 0 V (logisch **LOW**) oder die Spannung U_{Bit} (für logisch **HIGH**) an. Durch die spezielle Verschaltung von horizontal und vertikal verbundenen Widerständen werden diese Spannungen binär gewichtet zu einer Gesamtspannung U_a zusammengesetzt. Somit trägt jedes Bit seinen spezifischen Teil zur resultierenden Gesamtspannung bei.

Im Praktikum soll später ein 4Bit R2R Netzwerk aus den im Baukasten vorhandenen Bauteilen ausgebaut werden. Nutzen Sie für $2R$ Widerstände mit 100 k Ω und für R Widerstände mit 51 k Ω . Offensichtlich weichen die Werte 100 k Ω und 51 k Ω von einem idealen Verhältnis von $2R/R$ ab. Grundsätzlich gibt es natürlich auch 50 k Ω -Widerstände, jedoch entsprechen diese nicht der, für das Praktikum gewählten E6-Reihe. Diese Abweichung vom idealen Widerstandsverhältnis hat zur Folge, dass die Gewichtung der einzelnen Bits/Pins nicht mehr exakt binär verläuft. Andererseits liegt die Abweichung von 1 k Ω innerhalb der Toleranz der verwendeten Bauteile (entspricht bei beiden Widerständen 5 % vom Widerstandswert). Daher kann dieses Ungleichgewicht zwischen R und $2R$ für die Praktikumsaufgabe vernachlässigt werden.

Der Impedanzwandler

Wie schon in Abbildung 20 ersichtlich entspricht ein R2R-Netzwerk im allgemeinen einem Spannungsteiler. Wie später im Vorbereitungs-Kapitel 5.1 und auch allgemein in der Praxis werden R2R-Netzwerke meist als unbelasteter Spannungsteiler berechnet. Dies ist jedoch nur dann zulässig, wenn am Ausgang des Netzwerkes unendlich große oder zumindest sehr sehr große Widerstände als Last angeschlossen werden, da kleine Widerstände das Teilverhältnis beeinflussen würden.

Der Lautsprecher, der im Praktikum angesteuert werden soll, kann in einem Ersatzschaltbild durch einen Widerstand von ca. 8 Ω ersetzt werden. Würde man den Lautsprecher nun direkt an das R2R-Netzwerk hängen, so wäre der Lastwiderstand viel zu klein, sodass in der Berechnung der Ausgangsspannung des Netzwerkes und damit auch in der Berechnung der anzulegenden Bit-Werte, nicht mehr von einem unbelasteten Spannungsteiler ausgegangen werden kann. Daher wird eine besondere Schaltung eingesetzt um eine hochohmige Last am R2R-Netzwerk wirksam zu machen: der **Impedanzwandler**

Diese Schaltung, wie in Abbildung 21 ersichtlich, besteht im wesentlichen aus einem Operationsverstärker (OPV), dessen negativer Eingang mit dem Ausgang der Schaltung verbunden wird. Wie der Name schon vermuten lässt wurde dieses Bauteil in der Vergangenheit in analog aufgebauten Computern als Rechen-Baustein verwendet. Je nach äußerer Beschaltung des Bauteils wurden Rechenoperationen wie Additionen, Subtraktionen oder auch komplexere Funktionen wie Differentiation, Integration oder Logarithmierung implementiert. Heutzutage werden Operationsverstärker meist zur Verstärkung von Spannungen oder Strömen eingesetzt. Der detaillierte Aufbau bzw. die

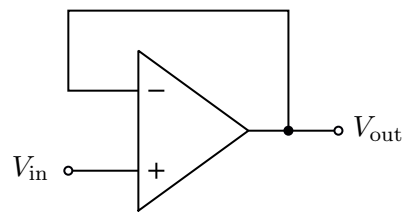


Abbildung 21: Verschaltung eines Operationsverstärkers (OPV) zu einem Impedanzwandler

Funktionalität eines solchen OPVs soll im weiteren nicht näher ausgeführt werden. Eine gute Erklärung der genauen Funktionsweise finden Sie bei Interesse beispielsweise unter www.electronicstutorials.ws/de/operationsverstarker/grundlagen.

Der Impedanzwandler stellt einen sehr hochohmigen Eingang zur Verfügung. Das am positiven Eingang anliegende Signal wird dabei 1:1 am Ausgang wieder ausgegeben. Damit eignet sich diese Schaltung gerade zu perfekt um den Lautsprecher an das R2R-Netzwerk anschließen zu können.

Richtige Verschaltung des Lautsprechers

Um den Lautsprecher nun in die Schaltung einfügen zu können werden noch 2 Bauteile benötigt:

- Koppelkondensator C_C :
Am Ausgang des DACs (des R2R Netzwerks) liegen Spannungen zwischen 0 V und 5 V an. Für eine richtige Funktionsweise benötigt der Lautsprecher jedoch eine Spannung zentriert um 0 V (also in unserem Fall von -2.5 V bis 2.5 V). Dies wird erreicht, in dem ein Kondensator zwischen OPV und Lautsprecher eingefügt wird. Im Praktikum soll dafür ein Kondensator mit $10\text{ }\mu\text{F}$ verwendet werden. Da es sich um einen Elektrolyt-Kondensator handelt, achten Sie auf die richtige Polarität! Der Ausgang des OPVs wird immer die höhere Spannung aufweisen, als am Lautsprecher.
- Dämpfungswiderstand R_D :
Würde man das Signal nach dem OPV und dem Kondensator direkt an den Lautsprecher legen, wäre der Ausgabeton sehr laut. Um die Lautstärke etwas abzuschwächen soll zwischen Kondensator und Lautsprecher noch ein Widerstand zur Dämpfung eingefügt werden. Als Widerstandswert sollen $220\text{ }\Omega$ verwendet werden.

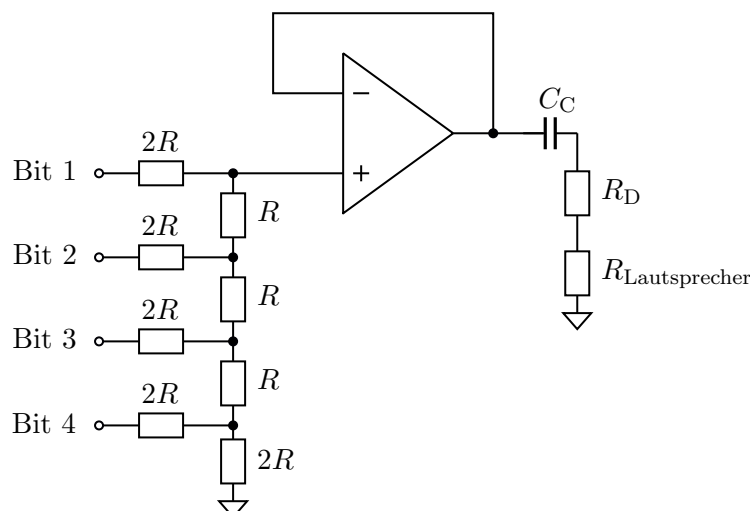


Abbildung 22: Gesamtschaltung der Lautsprecheransteuerung

Die Gesamtschaltung von R2R-Netzwerk, OPV, Kondensator, Widerstand und Lautsprecher ist in Abbildung 22 dargestellt. Der Lautsprecher kann dabei als Widerstand dargestellt werden.

5.1 Vorbereitung

Aufgabe 1:

- a) Gegeben sei ein als R2R-Netzwerk implementierter 2 Bit DAC. In Abbildung 23 ist die resultierende Schaltung ersichtlich. Berechnen Sie nun die Ausgangsspannung U_a für die folgenden Kombinationen an Eingangsspannungen:

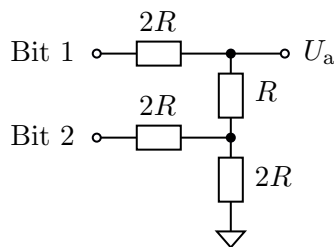


Abbildung 23: 2 Bit DAC

$U_{\text{Bit 1}}$	$U_{\text{Bit 2}}$	U_a
0 V	0 V	
0 V	U_{Bit}	
U_{Bit}	0 V	
U_{Bit}	U_{Bit}	

- b) Welche allgemeine Berechnungsvorschrift können Sie für die Spannung U_a ableiten?

$U_a =$ _____

- c) Erweitern Sie die Berechnungsvorschrift aus Aufgabe 1 für einen 4 Bit DAC:

$U_{a,4\text{Bit}} =$ _____

Aufgabe 2:

Im Praktikum soll später mit dem Mikrocontroller ein Sinus über das R2R Netzwerk in ein analoges Signal gewandelt werden.

- a) Berechnen Sie die benötigten Integer-Werte (ganzzahlig), sowie die einzelnen Bitvektoren (4 Bit in Hexadezimal-Darstellung), die Sie später am Eingang des DACs anlegen müssen, sodass sich am Ausgang ein Sinussignal (stufenförmiges Signal, das einer Sinuskurve folgt) zwischen 0 V und 5 V einstellt. Berechnen Sie dazu 20 Stützstellen einer Periode des Signals:

U_{SIN} (V)	Dezimal	Hexadezimal
2.5		
3.3		
4.0		
4.5		
4.9		
5.0		
4.9		
4.5		
4.0		
3.3		
2.5		
1.7		
1.0		
0.5		
0.1		
0.0		
0.1		
0.5		
1.0		
1.7		

- b) Die Periodendauer des Signals kann durch die Zeit, in der das Bitmuster an das Netzwerk angelegt wird eingestellt werden. Berechnen Sie wie lange ein Bitmuster für die folgenden Frequenzen (und damit Tonhöhen) anliegen muss:

Ton	Frequenz (Hz)	Zeit pro Bitmuster (μs)
c'	262	
d'	294	
e'	330	
f'	349	
g'	392	
a'	440	
h'	494	

5.2 Praktikumsaufgabe

Folgende Aufgabenschritte sollen während des Praktikums durchgeführt werden:

5.2.1 Aufbau der Schaltung:

- Bauen Sie am Steckbrett ein R2R-Netzwerk mit den im Baukasten vorhandenen Bauteilen auf.
- Bestimmen Sie 4 digitale Pins am Mikrocontroller, die zur Ausgabe der digitalen Daten verwendet werden sollen. Konfigurieren Sie die Pins bzw. den Port dementsprechend in der `setup()`-Funktion. Verbinden Sie die 4 Pins mit den Eingängen des R2R-Netzwerks am Steckbrett.
- Stecken Sie den Verstärker (OPV) auf das Steckbrett und verbinden Sie die Spannungsversorgungen V_{DD} und V_{SS} nach dem in Abbildung 24 angegebenen Schaltbild. Verbinden Sie anschließend die Eingänge des OPVs wie in Abbildung 22 dargestellt mit dem Ausgang des R2R-Netzwerks und mit dem Ausgang des OPVs selbst.

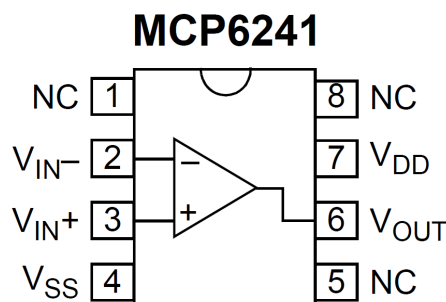


Abbildung 24: Pin-Konfiguration Operationsverstärkers MCP6241

- Schließen Sie den Koppelkondensator C_C sowie den Dämpfungswiderstand R_D an den Ausgang des OPVs an.

- e) Verbinden Sie als letzten Aufbauschritt den Lautsprecher mit dem Dämpfungswiderstand und dem Massepotential (GND).

5.2.2 Programmierung des Mikrocontrollers

- a) Programmieren Sie den Mikrocontroller so, dass er einen Sinus mit einer Frequenz von 440 Hz an den Lautsprecher ausgibt. Der Ton soll dabei nach jeweils 1 s durch eine 500 ms lange Pause unterbrochen werden.
- b) Erweitern Sie das Programm insofern, dass alle 7 verschiedenen Töne aus der Vorbereitung abgespielt werden können. Hinterlegen Sie die Tonfolge des bekannten Kinderliedes „Alle meine Entchen“ als Endlosschleife in Ihrem Programm und geben Sie das Lied am Lautsprecher aus.
- c) Die Töne, welcher der Mikrocontroller abspielt sollen in einem letzten Ausbauschritt des Programms nicht mehr direkt im Programm hinterlegt werden sondern vom Benutzer vorgegeben werden können. Programmieren Sie eine „Klaviertastatur“ über das serielle Interface des Mikrocontrollers. Die einzelnen Tasten der Tastatur des Praktikums-PCs sollen dabei wie in Abbildung 25 dargestellt mit den Tönen aus der Vorbereitung belegt werden.

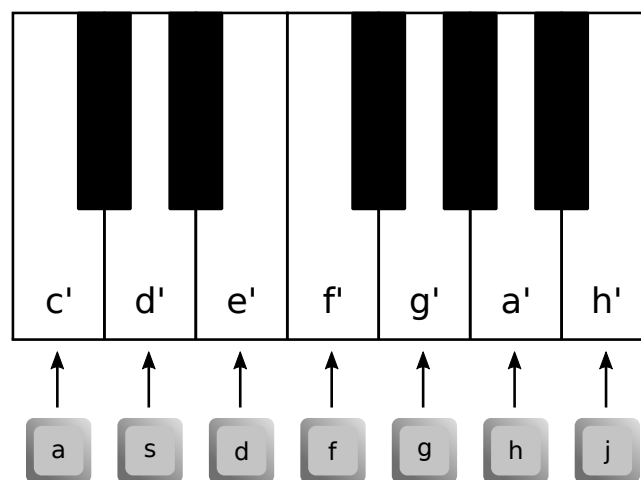


Abbildung 25: Tastenentsprechung der einzelnen Töne

- d) Testen Sie das fertige Programm am Mikrocontroller und zeigen Sie Ihre Ergebnisse dem Praktikumsleiter/Tutor bzw. der Praktikumsleiterin/Tutorin

5.3 Checkliste für das Protokoll

- ☐ Dokumentieren Sie Ihre Ergebnisse aus der Vorbereitung
- ☐ Erklären Sie den Aufbau Ihrer fertigen Schaltung (Skizze!). Welche Pins haben Sie für die Ausgabe am Mikrocontroller gewählt?
- ☐ Dokumentieren Sie Ihre Überlegungen zu den einzelnen Programmieraufgaben
- ☐ Halten Sie die Ergebnisse der Tests Ihrer Programme fest. Gibt es Verbesserungspotential in Ihrer Implementierung des „Mikrocontroller-Klaviers“?

6 Motorsteuerung mit Näherungssensor

In dieser Praktikumsaufgabe soll ein Motor mithilfe des Mikrocontrollers angesteuert werden. Damit soll eine Lüftersteuerung implementiert werden. Am Motor wurde dafür bereits ein kleiner Propeller montiert. Um sicher zu stellen, dass sich der Lüfter frei drehen kann, liegt im Baukasten eine Halterung für den Motor bei (siehe Abbildung 26).

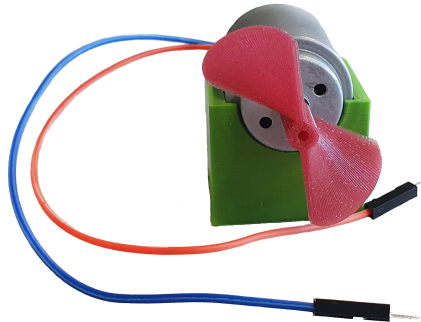


Abbildung 26: Motor mit Propeller in der Halterung

Bei dem verwendeten Motor handelt es sich um einen Gleichstrommotor wie er beispielsweise auch in PC-Lüftern eingebaut ist (in kleinerer Form). Der Motor kann mit ca. 5 V betrieben werden. Um genug Strom für den Motor liefern zu können wird ein Motortreiber verwendet. Der Treiber L293D ist als H-Brücke implementiert und ermöglicht es den Motor nach links und nach rechts laufen zu lassen. Der genaue Aufbau dieses Bausteins wird im Weiteren nicht näher erklärt, kann jedoch unter de.wikipedia.org/wiki/Vierquadrantensteller, bei Interesse, nachgelesen werden. Wichtig für den Betrieb des Motors ist die Funktion der H-Brücke, wie in Abbildung 27 dargestellt. Es werden immer zwei schräg gegenüberliegende Schalter gleichzeitig geschlossen. Soll der Motor sich nach rechts drehen, so müssen Schalter S1 und S4 geschlossen werden. Für einen linksläufigen Betrieb werden die Schalter S2 und S3 geschlossen.

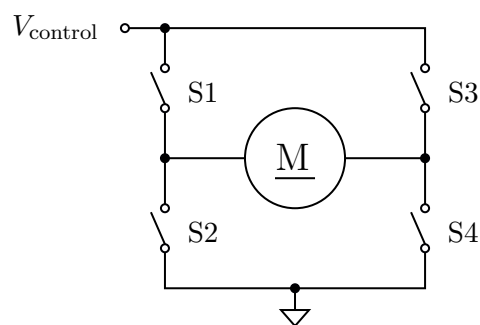


Abbildung 27: Vereinfachte Darstellung einer H-Brücke zur Ansteuerung eines Gleichstrommotors

Werden die beiden Schalterpaare zusätzlich gepulst (also kurz ein und dann wieder ausgeschaltet) betrieben, so kann die Drehzahl des Motors eingestellt werden. Man spricht dann von einer **Pulsweitenmodulation** der Schalter, oder auch PWM. Die entsprechenden Signale dafür können durch richtige Programmierung des Mikrocontrollers erzeugt werden.

Pulsweitenmodulation (PWM) mit dem Mikrocontroller

Bei der Pulsweitenmodulation wird das Puls-Pause-Verhältnis eines periodischen Signals bei konstanter Frequenz variiert. Das Verhältnis von Puls zu Pause wird dabei in % angegeben. In Abbildung 28 sind einige verschiedene Puls-Pausen-Verhältnisse für ein Rechtecksignal mit konstanter Frequenz ersichtlich (0 %, 25 %, 50 %, 75 % und 100 %). Sehen Sie sich dazu die Arduino[®]-Funktion `analogWrite()` an, www.arduino.cc/reference.

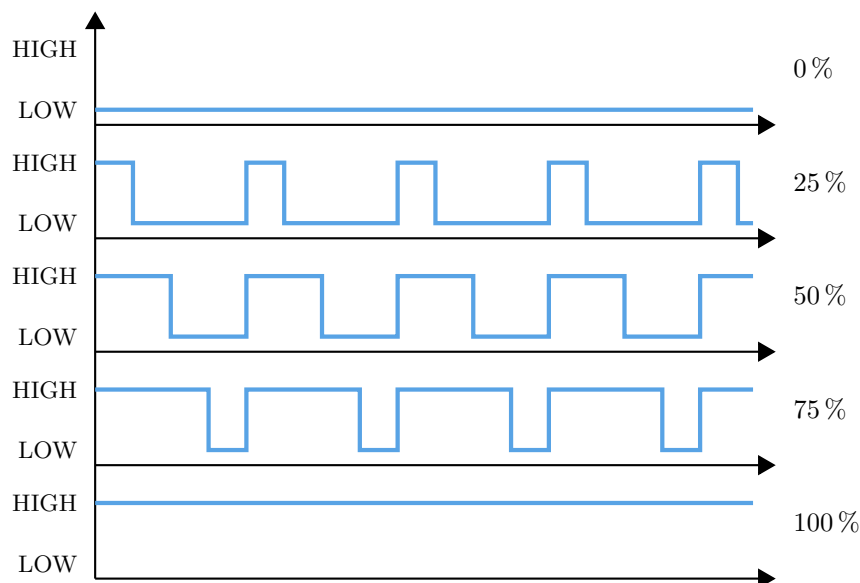


Abbildung 28: PWM Signale für verschiedene Puls-Pause-Verhältnisse

Motortreiber L293D

Wie oben bereits erwähnt wird für den Betrieb des Motors ein Treiber-Baustein benötigt um den Motor nach links und nach rechts drehen zu können und um genug Strom für den Motor liefern zu können. Als Baustein wird dabei der bereits erwähnte IC L293D verwendet. In Abbildung 29 ist die Pin-Konfiguration des Treibers ersichtlich.

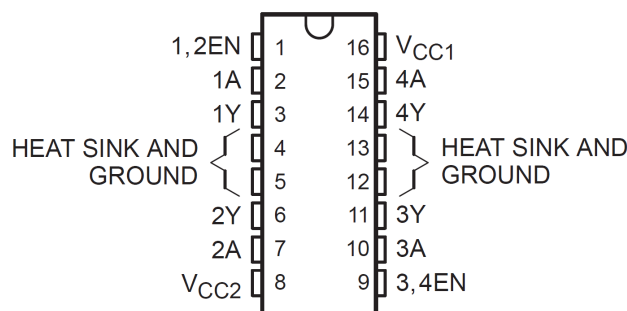


Abbildung 29: Pin-Konfiguration des Motortreibers L293D

Von den Pins des Bauteils wird nur die linke Hälfte (Pin 1 bis Pin 8) und Pin 16 benötigt. Die Pins sollen wie folgt beschalten und verwendet werden:

- 1,2EN – Enable Signal für die linke Pin-Seite des Bausteins. Dieser Pin sollte auf 5 V gelegt werden
- 1A/2A – Treibereingänge, hier wird später das Steuer-Signal (PWM-Signal für Rechts- und Linkslauf angelegt).

- 1Y/2Y – Treiberausgänge, zwischen diesen beiden Pins wird später der Motor angeschlossen.
- GROUND – Diese beiden Pins sollten mit GND verbunden werden.
- V_{CC1} und V_{CC2} – Spannungsversorgung für den Treiber-Baustein. Diese Pins sollten beide mit 5 V verbunden werden.

In der folgenden Tabelle ist die Funktion des Treibers noch einmal zusammengefasst. Anstatt der fixen Werte von HIGH und LOW können wie bereits beschrieben auch PWM-Signale verwendet werden.

EN	1A	2A	Funktion
HIGH	LOW	HIGH	Rechtslauf
HIGH	HIGH	LOW	Linkslauf
HIGH	HIGH	HIGH	Schnell-Halt
LOW	X	X	Freilauf

Näherungssensoren und Anzeige der Drehzahlstufe

Zusätzlich zum reinen Betrieb des Motors sollen noch zwei weitere Komponenten implementiert werden:

- Abstandsmessung mit Ultraschall-Modulen

Mithilfe eines Ultraschall-Sensors soll ein Not-Halt des Lüfters realisiert werden. Nähert sich eine Person dem drehenden Lüfter auf weniger als 30 cm, soll der Motor abstoppen.

Ein Ultraschall-Sensor sendet zum Messen des Abstandes ein Ultraschall-Signal aus. Trifft dieses auf ein Hindernis, so wird das Signal reflektiert und kann am Empfänger detektiert werden. Um den Abstand d zu bestimmen wird die Laufzeit T_s des Signals gemessen. Der Abstand kann bestimmt werden zu:

$$d = \frac{v \cdot T_s}{2}$$

Die Variable v soll in der oben angeführten Formel die Schallgeschwindigkeit im entsprechenden Medium/Material darstellen. Für Luft beträgt die Schallgeschwindigkeit $v_{\text{Luft}} = 344 \text{ m/s}$.



Abbildung 30: Ultraschall-Modul HC-SR04

Das in Abbildung 30 dargestellte Modul eignet sich dazu Entfernungen zwischen 2 cm und ca. 3 m mit einer Auflösung von 3 mm zu messen. Folgende 4 Pins (von links nach rechts in der Abbildung) werden verwendet:

- V_{CC} $\Rightarrow$ Versorgungsspannung von 5 V
- Trig $\Rightarrow$ Eine fallende Flanke am Triggereingang für mindestens 10 μ s startet den Messzyklus. Ein Messzyklus kann alle 20 ms gestartet werden
- Echo $\Rightarrow$ Nach dem Starten des Messzyklus ist der Echo-Pin für die Dauer der Signallaufzeit auf HIGH
- GND $\Rightarrow$ Masseanschluss auf 0 V

• Drehzahlanzeige an einer 7-Segment-Anzeige

Um die Geschwindigkeit, mit der der Motor sich gerade dreht zu visualisieren, soll die Drehzahlstufe des Motors auf einer 7-Segment-Anzeige ausgegeben werden. Eine 7-Segment-Anzeige besteht in der Regel aus 8 LEDs, welche an jeweils einem Anschluss (Kathode oder Anode) zusammengeschlossen sind. In Abbildung 31 ist die Pinbelegung, sowie das Ersatzschaltbild einer solchen Anzeige ersichtlich.

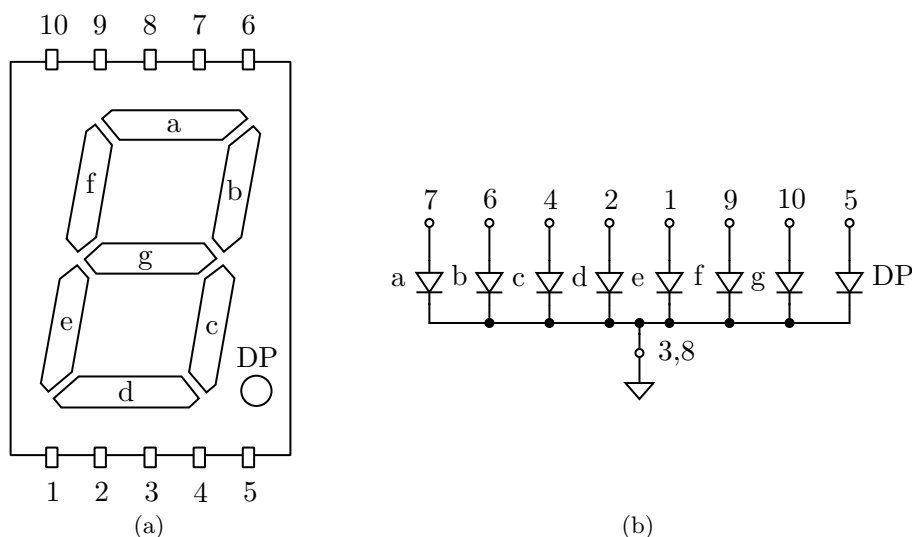


Abbildung 31: Pinbelegung und Ersatzschaltbild der verwendeten 7-Segment-Anzeige SC52

Im Praktikum wird eine 7-Segment-Anzeige mit Common-Cathode Anschluss verwendet (siehe Abbildung 31b). Für die 8 Segmente (7 Segmente der Ziffer und 1 Dezimalpunkt) wird jeweils ein Vorwiderstand benötigt um das Bauteil vor zu hohen Spannungen und Strömen zu schützen.

Um Pins am Mikrocontroller zu sparen und nicht jedes der 8 Segmente einzeln mit einem Pin verbinden zu müssen wird ein Schieberegister verwendet. Im Baukasten steht Ihnen ein Schieberegister des Typs CD74HC595E zur Verfügung. Dieser Baustein gibt 8 Bit an seriell angelegten Daten auf 8 parallelen Ausgängen aus. In Abbildung 32 ist die Pinbelegung des Schieberegisters ersichtlich.

Die einzelnen Pins sind wie in der nachfolgenden Tabelle beschrieben anzusteuern.

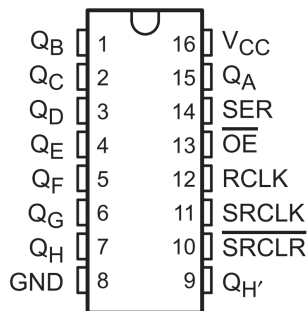


Abbildung 32: Pinbelegung

Pin	Funktion
V _{CC}	Versorgungsspannung (5 V)
GND	Masseanschluss (0 V)
SER	Serieller Dateneingang
$\overline{OE}$	Output Enable Eingang, ein LOW an diesem Pin gibt die Ausgänge frei
RCLK	Register-Clock, ein LOW zu HIGH Impuls überträgt die Daten im Schieberegister auf die Ausgänge
SRCLK	Shift-Register-Clock, ein LOW zu HIGH Impuls überträgt das Bit an Pin SER in das Schieberegister
$\overline{SRCLR}$	Low active reset, Ein LOW-Signal an diesem Eingang setzt die Ausgänge des Schieberegisters auf LOW
Q _{H'}	Überlauf
Q _A - Q _H	Ausgänge des Schieberegisters

Nützliche Arduino® Funktionen

Folgende zusätzliche Funktionen, die bereits im Arduino®-Framework vordefiniert sind, können bei dieser Praktikumsaufgabe nützlich sein:

Befehl	Erklärung
<code>pulseIn(pin, value)</code>	Gibt die Dauer eines Pulses zurück der an einem definierten digitalen Pin des Mikrocontrollers anliegt. Dabei wird ein Timer gestartet sobald am zuvor definierten Pin ein Spannungspegel High oder LOW anliegt. Auf welches Logiklevel der Mikrocontroller reagiert wird mit der Variable <code>value</code> definiert. Der Pin, an dem der Puls detektiert werden soll wird mit <code>pin</code> festgelegt. Nach dem Start des Timers wartet der Mikrocontroller bis der Puls durch eine fallende oder steigende Flanke beendet ist. Anschließend gibt die Funktion die Länge des Impulses in Mikrosekunden zurück. Die Funktion funktioniert mit Impulsen der Länge von 10 Mikrosekunden bis 3 Minuten.
<code>shiftOut(dataPin, clockPin, bitOrder, value)</code>	Shiftet ein Byte von Daten seriell über einen Pin hinaus. <code>dataPin</code> gibt den Pin an, welcher die seriellen Daten ausgibt. <code>clockPin</code> gibt den Pin an, welcher den seriellen Clock ausgibt. <code>bitOrder</code> gibt an, ob mit dem MSB oder LSB begonnen wird (gültige Werte: <code>MSBFIRST</code> oder <code>LSBFIRST</code>). <code>value</code> ist das Byte, welches rausgeschiftet werden soll (0-255). Wichtig: die beiden Pins <code>dataPin</code> und <code>clockPin</code> müssen händisch als Ausgang konfiguriert werden.

Nähere Informationen zu den oben genannten Funktionen unter www.arduino.cc.

6.1 Vorbereitung

Aufgabe 1:

Informieren Sie sich über die richtige Verwendung und Ansteuerung des Ultraschall-Moduls. Benutzen Sie die unter www.mikrocontroller.net angeführten Informationen zum Ultraschall-Modul HC-SR04 um ein Konzept zur Abstandsmessung für die Praktikumsaufgabe zu entwickeln.

Aufgabe 2:

Überlegen Sie welche Bytes Sie an das Schieberegister übertragen müssen, um die Ziffern von 0-9 an der 7-Segmentanzeige darstellen zu können:

	A	B	C	D	E	F	G	DP
0								
1								
2								
3								
4								
5								
6								
7								
8								
9								

Aufgabe 3:

Die benötigte Diodenspannung der LEDs an der 7-Segmentanzeige beträgt ca. $U_D = 2\text{ V}$ bei einem Strom von $I_D = 15\text{ mA}$. An den Ausgängen des Schieberegisters ist mit einer Spannung von 5 V für ein logisches HIGH-Signal zu rechnen. Berechnen Sie den benötigten Vorwiderstand R_V für die einzelnen LEDs der 7-Segmentanzeige und wählen Sie einen geeigneten Widerstandswert aus der E6 Reihe als Bauteil aus:

Berechnet:

$$R_{V,\text{calc}} = \underline{\hspace{2cm}} \Omega$$

Aus E6-Reihe gewählt:

$$R_V = \underline{\hspace{2cm}} \Omega$$

6.2 Praktikumsaufgabe

Implementieren Sie die Lüftersteuerung mit Nothalt. Der Motor/Lüfter soll dabei stoppen, sobald sich etwas/jemand weniger als 30 cm den Rotorblättern nähert. Die Drehzahl soll mithilfe von 2 Tastern geregelt werden können. Mit Hilfe der 7-Segmentanzeige soll die aktuelle Drehzahlstufe angezeigt werden. Führen Sie für diese Praktikumsaufgabe die folgenden Schritte durch:

- a) Realisieren Sie zunächst die Anzeige an der 7-Segmentanzeige. Verbinden Sie die Anzeige mit den Vorwiderständen und dem Schieberegister und stellen Sie eine Verbindung zum Mikrocontroller her. Implementieren Sie die Ansteuerung des Schieberegisters.
- b) Testen Sie die Funktion der 7-Segmentanzeige indem Sie mithilfe der beiden Taster die Ziffern von 0-9 nach oben und nach unten durchschalten. Verwenden Sie zum Einlesen der Taster externe Interrupts.
- c) Platzieren Sie den Motor und den Motortreiber am Steckbrett. Verbinden Sie den Motor mit dem Treiber und testen Sie die Funktion des Motors, indem Sie die Eingänge des Treiber-Bausteins zuerst direkt mit 5 V und GND (0 V) verbinden.
- d) Implementieren Sie die Drehzahl-Steuerung des Motors mithilfe einer PWM am Mikrocontroller. Der Motor soll sich zunächst nach links drehen. Mithilfe eines Tasters soll es möglich sein, die Drehzahl des Motors zu erhöhen. Sehen Sie dazu 10 verschiedene Drehzahlstufen vor (Stufe 1 bis 9 + Stillstand). Wird der Taster in der höchsten Drehzahlstufe erneut gedrückt, so soll wieder mit der niedrigsten Drehzahl begonnen werden. Zusätzlich sollen 2 LEDs verwendet werden um anzuzeigen, ob der Motor eingeschaltet ist oder nicht. Verwenden Sie dazu eine grüne und eine rote LED aus dem Baukasten. Läuft der Motor, so soll die grüne LED leuchtet. Ist der Motor im Stillstand, so soll die rote LED eingeschaltet werden.
- e) Implementieren Sie die Ansteuerung und Auswertung des Ultraschall-Sensors am Mikrocontroller. Testen Sie Ihre Realisierung, indem Sie die berechnete Entfernung über den Serial Monitor am PC ausgeben. Überprüfen Sie die Umrechnung, indem Sie Gegenstände oder ähnliches in unterschiedlichen Abständen vor dem Sensor platzieren und den Abstand mit einem Lineal nachmessen.
- f) Fügen Sie nun 7-Segmentanzeige, Taster, LEDs, Motor und Abstandssensor zusammen. Die fertige Schaltung und das dazugehörige Programm sollen folgende Funktionen erfüllen:
 - Bei Betätigung des rechten Tasters soll die Drehgeschwindigkeit des Motors nach rechts erhöht werden (sollte sich der Motor zuvor nach Links gedreht haben, ändert der Motor bei Tastendruck die Richtung)
 - Bei Betätigung des linken Tasters soll die Drehgeschwindigkeit des Motors nach links erhöht werden (sollte sich der Motor zuvor nach Rechts gedreht haben, ändert der Motor bei Tastendruck die Richtung)
 - Bei einer Annäherung eines Gegenstandes oder einer Person auf weniger als 30 cm zum Motor, soll der Motor sofort angehalten werden. Entfernt sich die Person oder der Gegenstand wieder weiter als 30 cm aus dem Gefahrenbereich soll der Motor mit selber Geschwindigkeit und Richtung wie zuvor weiterlaufen.
 - Steht der Motor still, soll die rote LED leuchten. Ist der Motor eingeschaltet und dreht sich, soll die grüne LED leuchten

6.3 Checkliste für das Protokoll

- ☐ Dokumentieren Sie Ihre Ergebnisse aus der Vorbereitung.
- ☐ Erklären Sie Ihre Überlegungen zur Ansteuerung des Ultraschall Moduls.
- ☐ Beschreiben Sie Ihre Vorgehensweise zu den einzelnen Aufgaben-Schritten während des Praktikums. Dokumentieren Sie dazu auch den Code zu den einzelnen Schritten.
- ☐ Dokumentieren Sie ausführlich die letzte Ausbaustufe der Schaltung und des Programms.

7 Datenübertragung mittels Infra-Rot (IR)

Infrarot-Datenübertragung wird im Alltag vielfach in Fernbedienungen jeder Art eingesetzt. Das Übertragungsverfahren beruht auf der Signal-Übertragung durch Licht im infraroten Spektrum (Wellenlängen zwischen 780 nm und 1 mm). In der folgenden Praktikumsaufgabe sollen mithilfe einer Infrarot-LED durch Morse-Code Daten zwischen 2 Mikrocontrollern übertragen werden.

Infrarot-LED

Eine Infrarot-LED (IR-LED) ist eine spezielle Form der Leuchtdiode. Mithilfe dieses Bauteils ist es möglich Licht im nahinfraroten Bereich des Spektrums zu emittieren. Die Wellenlänge solcher IR-LEDs liegt zwischen 700 nm und 1 µm. Die im Praktikum verwendete Diode emittiert Licht mit einer Wellenlänge von 940 nm, also ein Signal außerhalb des für den Menschen sichtbaren Lichtspektrums.

Die im Praktikum verwendete IR-Diode soll bei einem Diodenstrom von 130 mA und einer Diodenspannung von 1.6 V betrieben werden. Der Mikrocontroller kann allerdings an den Ausgangspins nur je 20 mA liefern. Daher wird ein MOS-Transistor verwendet um genügend Strom für die IR-Diode bereitstellen zu können. Die zu verwendende Schaltung ist in Abbildung 33 dargestellt:

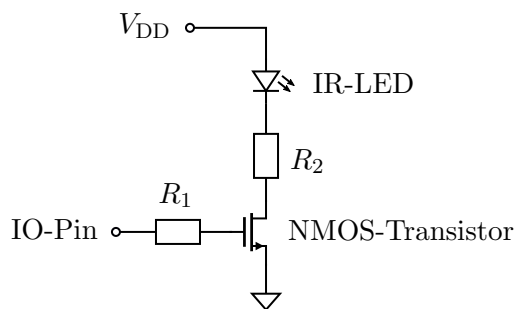


Abbildung 33: Beschaltung des Mikrocontrollers und der IR-LED

Mithilfe des digitalen Ausgangspins des Mikrocontrollers (IO-Pin) wird der Transistor ein- (logisch **HIGH**) und ausgeschaltet (logisch **LOW**). Die Diode mit ihrem Vorwiderstand wird zwischen die Versorgungsspannung von 5 V den Transistor und GND (0 V) gehängt. Schaltet der Transistor bei einem **HIGH**-Signal am Gate durch, so kann genug Strom durch die Diode fließen, da der 5 V-Pin am Mikrocontroller-Board 250 mA liefern kann. Der Vorwiderstand R_2 ist notwendig um den Strom durch die Diode zu begrenzen, damit die zulässige maximale Strombelastung des Bauteils nicht überschritten wird. Auch am Gate des Transistors wird ein Widerstand benötigt. Wird an den Transistor am Gate ein **HIGH**-Signal angelegt (um den Transistor einzuschalten), so verhält sich im ersten Moment der Gate-Anschluss des Transistors wie ein Kondensator. Sobald diese Kapazität aufgeladen ist, ist auch der Transistor voll durchgeschaltet. Würde man nun den Transistor direkt an den Mikrocontroller anschließen, und den Pin auf **HIGH** schalten, würde sich der Kondensator über einen nahezu unendlich kleinen Widerstand in minimaler Zeit aufladen. Setzt man diese Annahmen in die Lade-Formel für einen Kondensator ein ($R \ll \tau$)

$$i_C(t) = \frac{U_0}{R} \cdot e^{-t/\tau}$$

so ergibt sich ein nahezu unendlich hoher Strom der im ersten Moment vom Pin des Mikrocontrollers geliefert werden müsste. Um den Mikrocontroller vor diesem hohen Stromfluss zu schützen wird ein Widerstand von $R_1 = 100 \Omega$ in Serie zwischen Mikrocontroller-Pin und Transistor-Gate geschaltet. Über diesen Widerstand kann sich die Gate-Kapazität etwas langsamer, aber dafür mit einem begrenzten Ladestrom aufladen.

Infrarot-Detektor

Um das von der IR-LED ausgesandte Signal empfangen und dekodieren zu können wird ein Empfangsmodul benötigt, das das IR-Signal wieder in ein elektrisches Signal umwandelt. Dabei wird für das Praktikum das IR-Detektor-Modul KY-022 verwendet (siehe Abbildung 34).

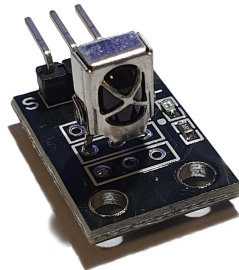


Abbildung 34: IR-Detektor-Modul KY-022

Detektiert dieses Modul ein infrarotes Signal mit einer Frequenz von 38 kHz, so schaltet der Ausgang des Moduls auf **LOW** solange das entsprechende IR-Signal empfangen wird. Ansonsten liegt am Ausgang dieses Moduls ein **HIGH**-Signal an. Mithilfe der Funktion `pulseIn(pin, value)` kann die Länge des vom Empfangs-Modul ausgegebenen Impulses am Mikrocontroller gemessen werden.

7.1 Vorbereitung

Aufgabe 1:

Um Daten mithilfe der Infrarot-LED übertragen zu können, wird eine Codierung und Modulation des 38 kHz-Signals benötigt. Eine einfache und schon seit langem im Einsatz befindliche Codierungsmethode ist das Morse-Alphabet. Die Morsezeichen gehen auf den amerikanischen Erfinder Samuel Morse (1833) zurück und bestehen aus insgesamt 3 Symbolen: kurzes Signal, langes Signal und Pause. Dabei wird ein direktes Signal (in Fall der Praktikumsaufgabe das 38 kHz IR-Signal) ein- und ausgeschaltet. Recherchieren Sie die Codierung des Alphabets. Welche Symbol- bzw. Zeichenfolge wird für die einzelnen Buchstaben verwendet?

Buchstabe	Code	Buchstabe	Code	Buchstabe	Code
A		J		S	
B		K		T	
C		L		U	
D		M		V	
E		N		W	
F		O		X	
G		P		Y	
H		Q		Z	
I		R			

Aufgabe 2:

Überlegen Sie, wie eine Codierung des 38 kHz-Signals mithilfe des Mikrocontrollers durchgeführt werden könnte, die zu sendenden Wörter/Buchstaben sollen dabei über die serielle Schnittstelle eingegeben werden können. *Hinweis:* Recherchieren Sie dazu die richtige Verwendung der Funktion `tone(pin, frequency, duration)`. Wie kann diese Funktion für die Aufgabenstellung eingesetzt werden?

Aufgabe 3:

Wie zuvor beschrieben und in Abbildung 33 dargestellt, soll später im Praktikum die IR-LED mithilfe eines Vorwiderstandes und eines Transistors betrieben werden. Durch die Diode soll dabei ein Strom von $I_D = 130 \text{ mA}$ fließen. Die Diodenspannung beträgt $U_D = 1.6 \text{ V}$. Aus dem Datenblatt kann abgelesen werden, dass der On-Widerstandes des Transistors R_{on} ca. 14Ω entspricht. Berechnen Sie aus diesen Informationen den Vorwiderstand der LED:

$R_2 =$ _____

7.2 Praktikumsaufgabe

Es soll ein Infrarot-Chat zwischen zwei Mikrocontrollern implementiert werden. Dafür wird Ihnen zusätzlich zu dem im Baukasten vorhandenen Mikrocontroller ein weiterer baugleicher Arduino®-Clone zur Verfügung gestellt. Führen Sie die folgenden Schritte durch:

- Bauen Sie die Beschaltung der IR-LED (bestehend aus den Widerständen R_1 und R_2 , dem Transistor und der IR-LED selbst) am Steckbrett auf.
- Realisieren Sie im Code die Eingabe von Buchstaben über die serielle Schnittstelle. Die eingegebenen Daten sollen per Morse-Signal über die IR-LED gesandt werden.
- Verbinden Sie das IR-Empfangsmodul mit dem zweiten Mikrocontroller. Entwickeln Sie ein Mikrocontroller-Programm, dass die empfangenen Daten am Modul einliest und über die serielle Schnittstelle am PC ausgibt. Zur Anzeige der gesendeten Daten soll mit dem Programm *HTerm* erfolgen, da immer nur ein Serial Monitor innerhalb der Arduino®-IDE geöffnet werden kann (ein Serial Monitor wird ja zum Senden der Daten benötigt). *HTerm* ist ein Terminal-Programm mit dem unter Windows® die serielle Schnittstelle angesprochen werden kann.

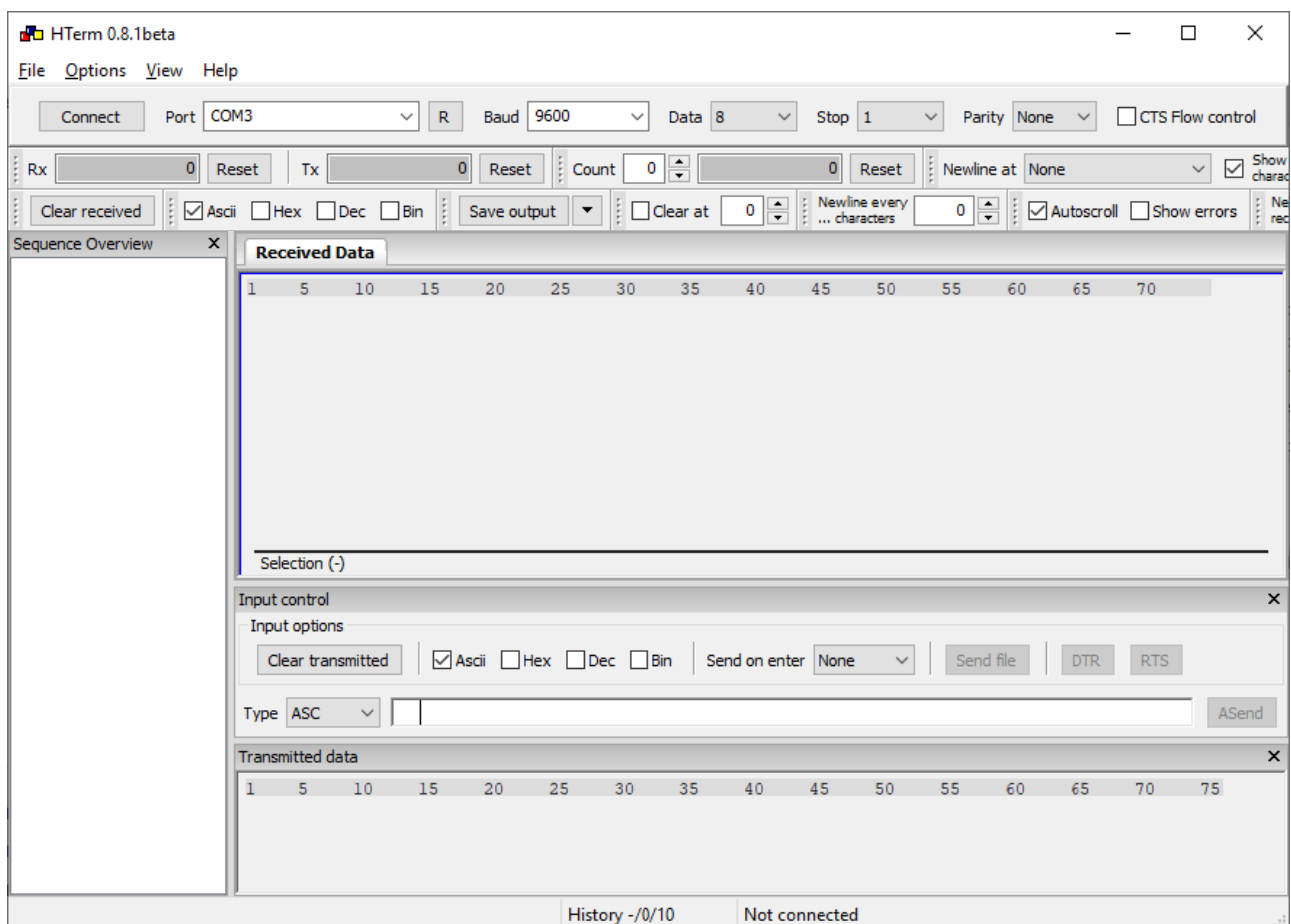


Abbildung 35: Benutzeroberfläche von *HTerm*

In Abbildung 35 ist die Benutzeroberfläche von *HTerm* ersichtlich. Folgende Einstellungen müssen getroffen werden, um die Daten des Mikrocontrollers einzulesen und anzuzeigen:

- Baudrate = 9600 (bzw. dieselbe Rate, mit der der Mikrocontroller die Daten an der seriellen Schnittstelle ausgibt)
- Port = richtige COM-Port des IR-Empfängers

- Um sich mit dem Mikrocontroller zu verbinden, muss zu Beginn der Übertragung noch auf *Connect* geklickt werden.
- d) Testen Sie die Datenübertragung zwischen Empfänger und Sender mit verschiedenen Wörtern bzw. Buchstabenreihen.

7.3 Checkliste für das Protokoll

- ☐ Dokumentieren Sie Ihre Ergebnisse aus der Vorbereitung.
- ☐ Beschreiben Sie die Ergebnisse der Oszilloskop-Station. Welche Beobachtungen konnten Sie im Bezug auf das vom Mikrocontroller erzeugten Signals machen?
- ☐ Erklären Sie Ihre Vorgehensweise zu den einzelnen Aufgaben-Schritten während des Praktikums. Dokumentieren Sie dazu auch den Code zu den einzelnen Schritten.
- ☐ Dokumentieren Sie die durchgeführten Tests der vollständig aufgebauten Datenübertragungsstrecke.